

Generative AI using Large Language Models (LLMs)

Lecture 8 | 03/11/25

Instructor: Chen Wang
Fri 4:10 pm - 6:40 pm | 517 Hamilton Hall



Agenda

Instruction Finetuning

Single Task Fine Tuning

Multi-task, Instruction Finetuning

Parameter Efficient Finetuning (PEFT)

Low-Rank Adaptation of Large Language Models (LoRA)

Soft Prompt Tuning



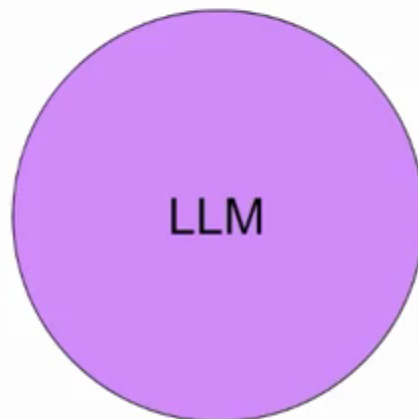
Finetuning an LLM with instruction prompts

In-context learning (ICL) - zero shot inference

Prompt

Classify this review:
I loved this DVD!
Sentiment:

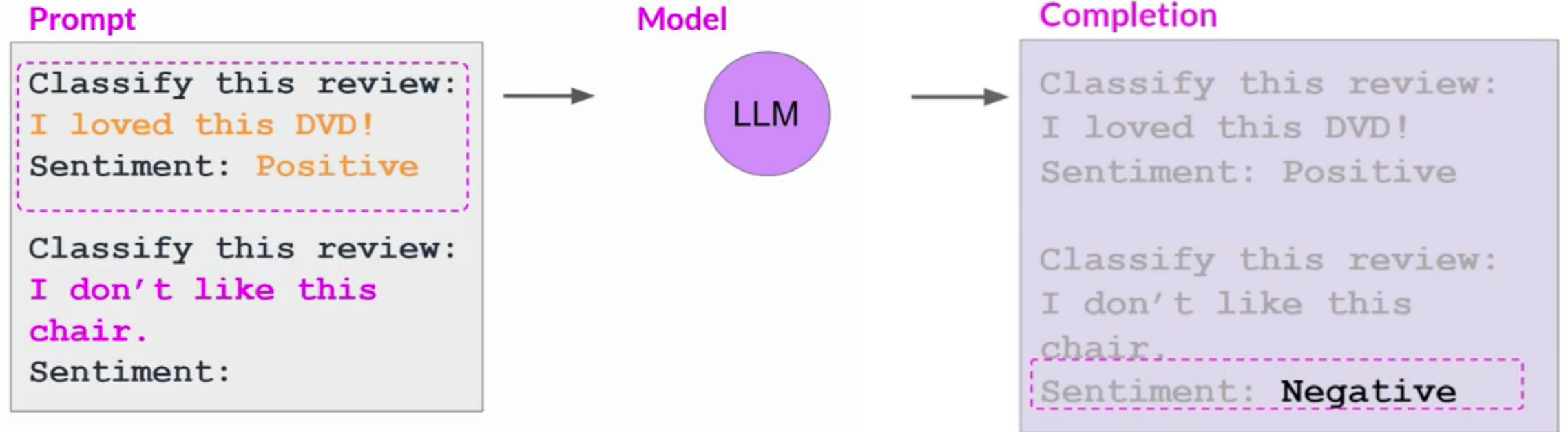
Model



Completion

Classify this review:
I loved this DVD!
Sentiment: eived a
very nice book review

In-context learning (ICL) - one/few shot inference



One-shot or Few-shot Inference

Limitations of in-context learning

Classify this review:

I loved this movie!

Sentiment: **Positive**

Classify this review:

I don't like this chair.

Sentiment: **Negative**

Classify this review:

This sofa is so ugly.

Sentiment: **Negative**

Classify this review:

Who would use this product?

Sentiment:

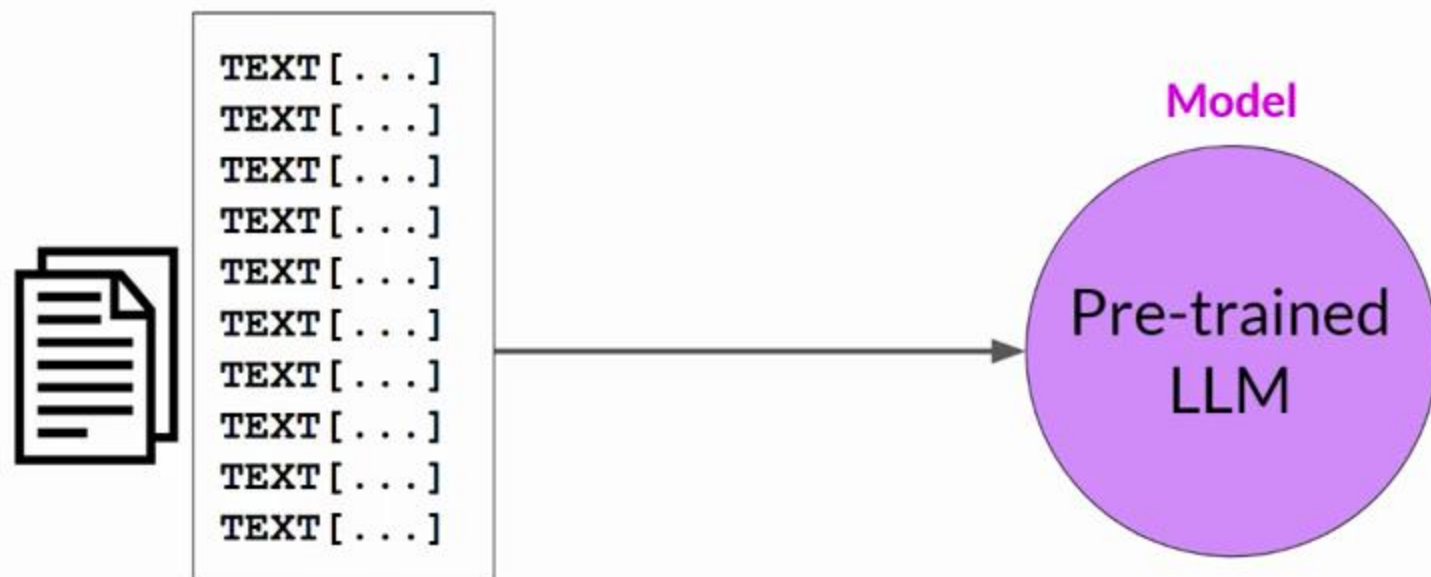
Even with
multiple
examples

- In-context learning may not work for smaller models **LLM**
- Examples take up space in the context window

Instead, try **fine-tuning** the model

LLM fine-tuning at a high level

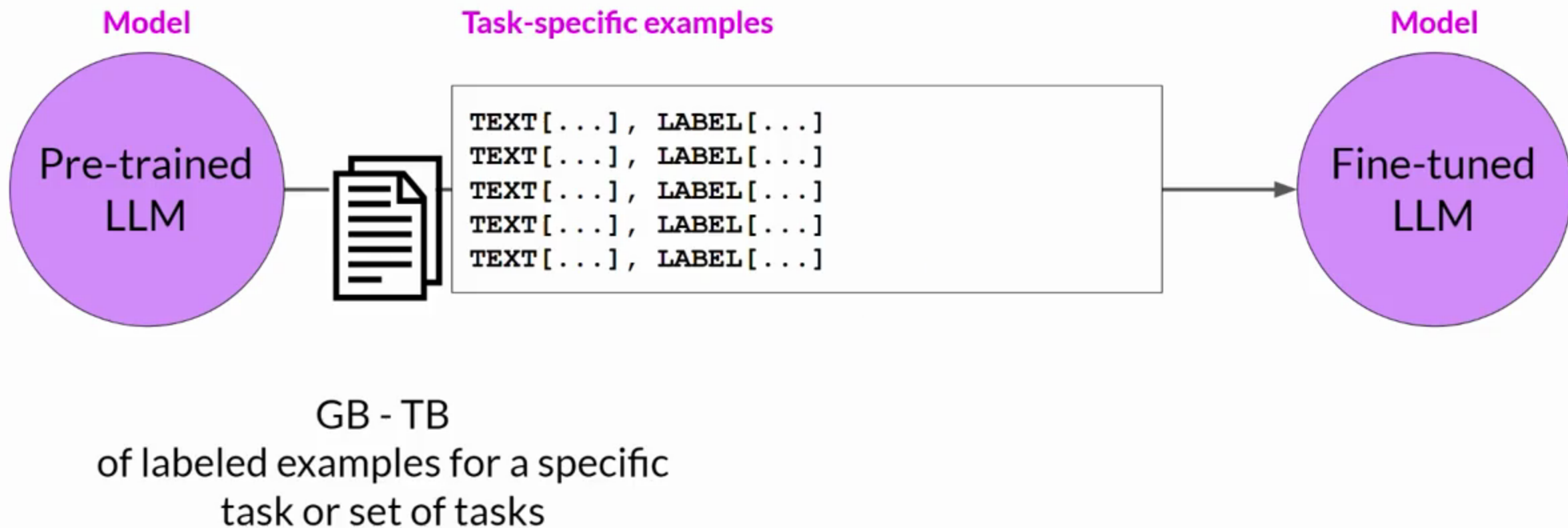
LLM pre-training



GB - TB - PB
of unstructured textual data

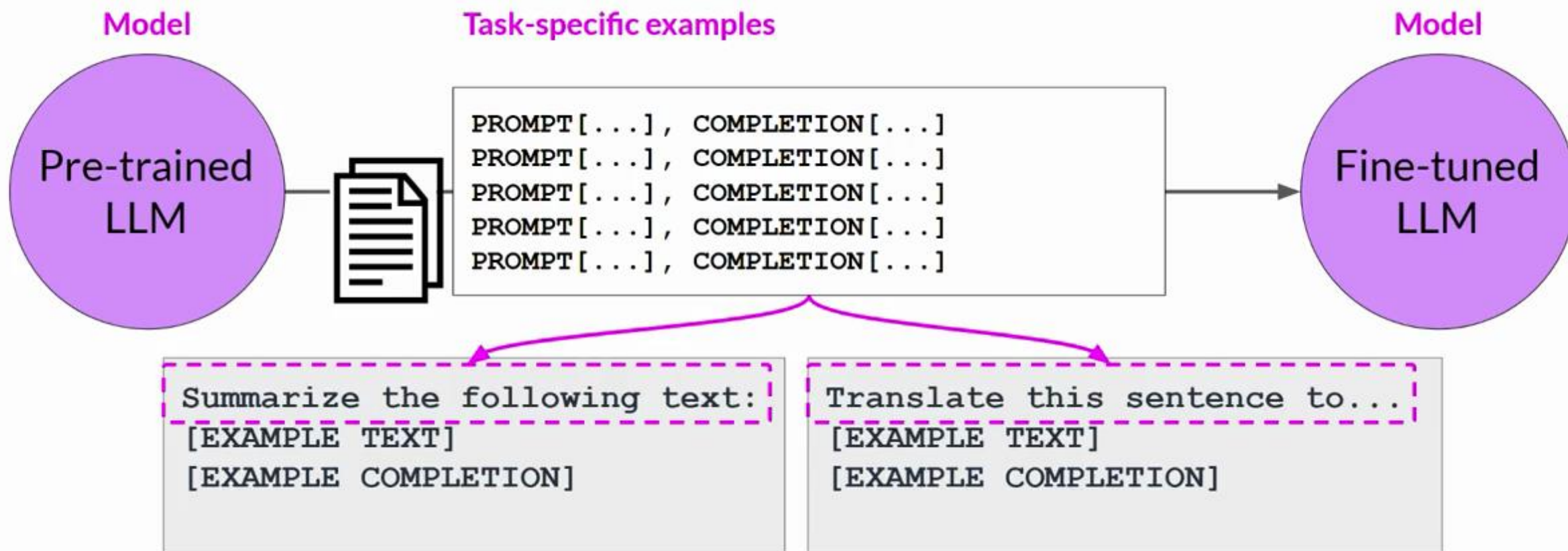
LLM fine-tuning at a high level

LLM fine-tuning



Using prompts to fine-tune LLMs with instruction

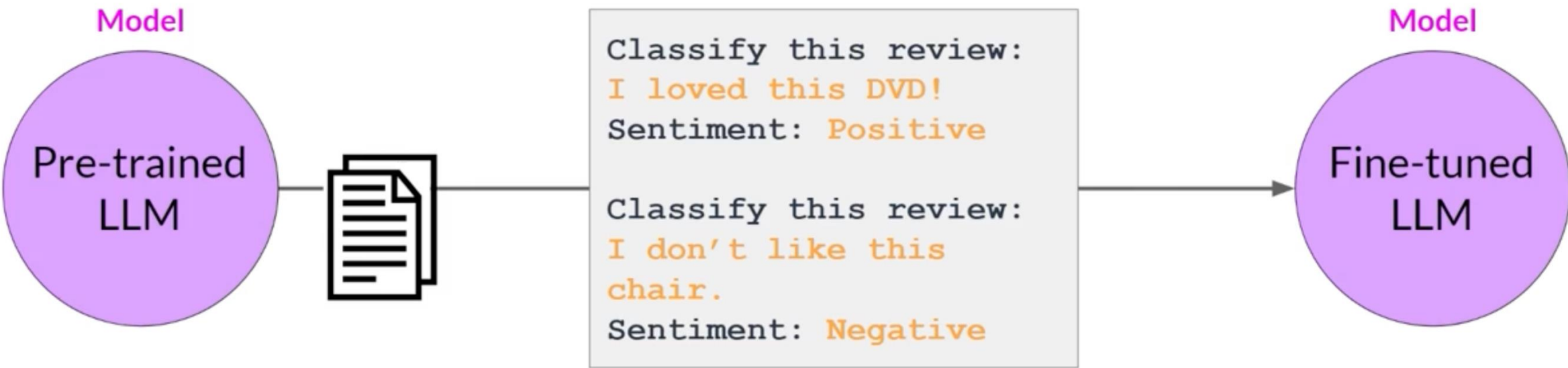
LLM fine-tuning





Using prompts to fine-tune LLMs with instruction

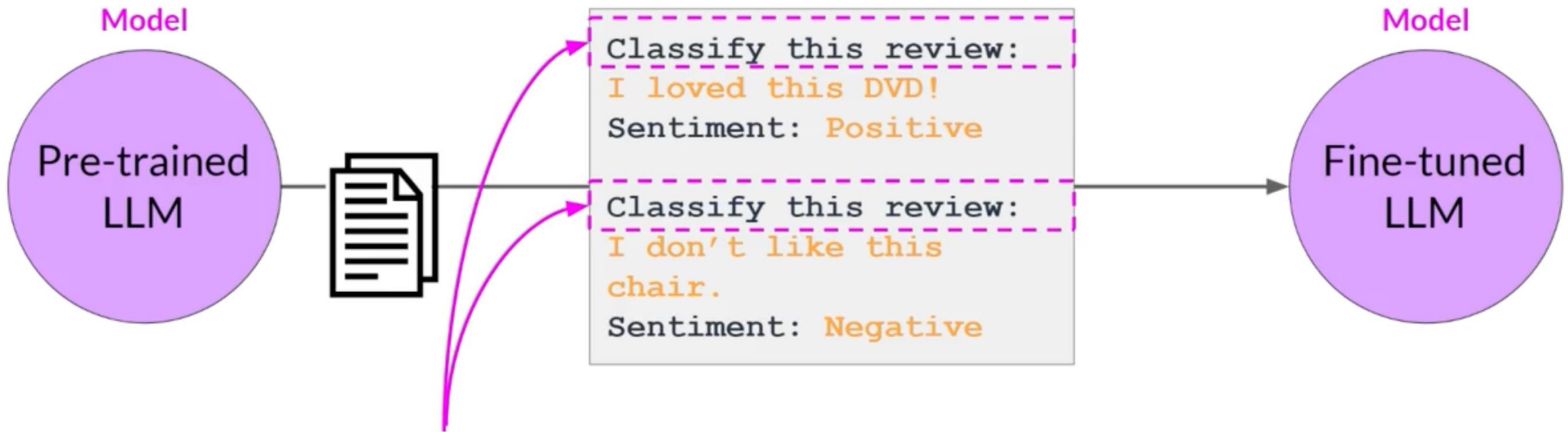
LLM fine-tuning





Using prompts to fine-tune LLMs with instruction

LLM fine-tuning

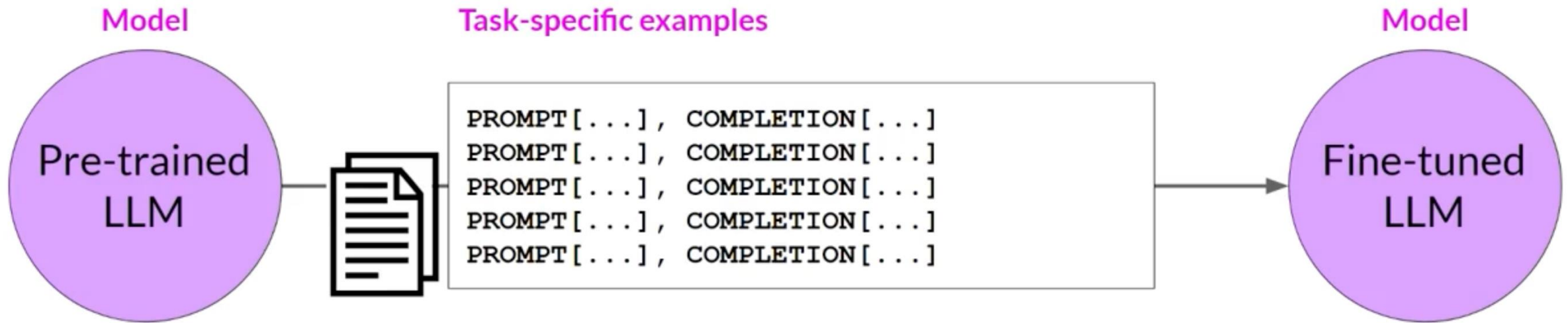


Each prompt/completion pair includes a specific "instruction" to the LLM



Using prompts to fine-tune LLMs with instruction

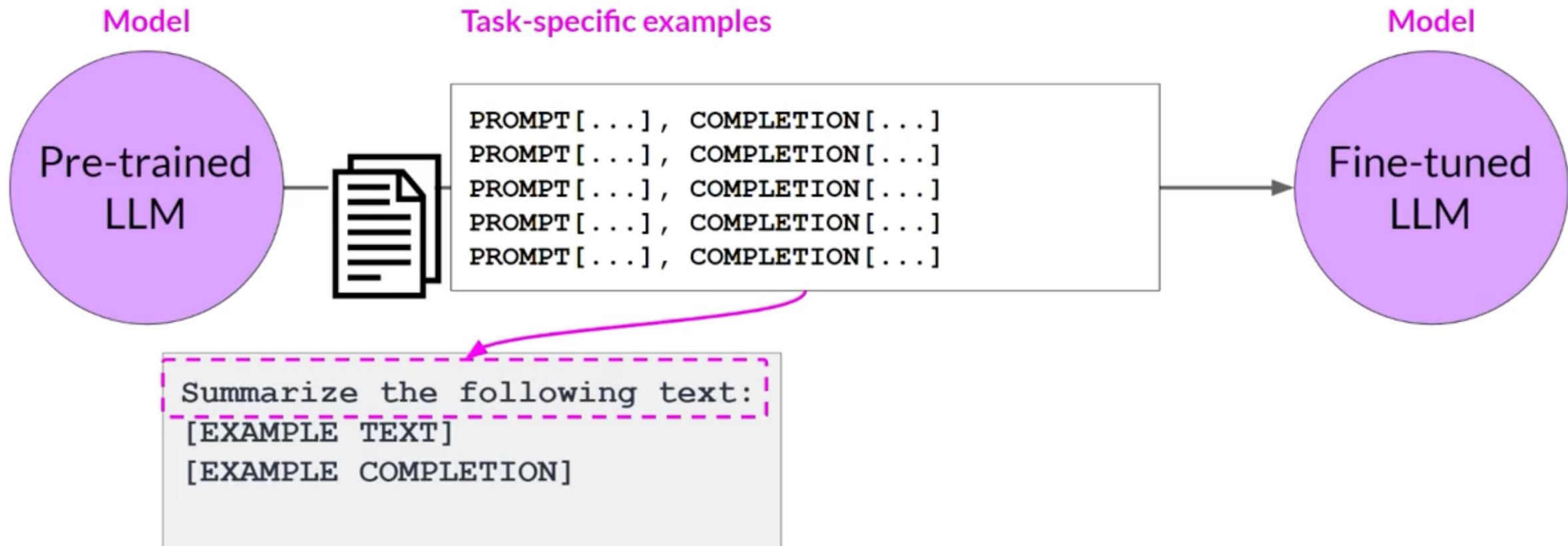
LLM fine-tuning





Using prompts to fine-tune LLMs with instruction

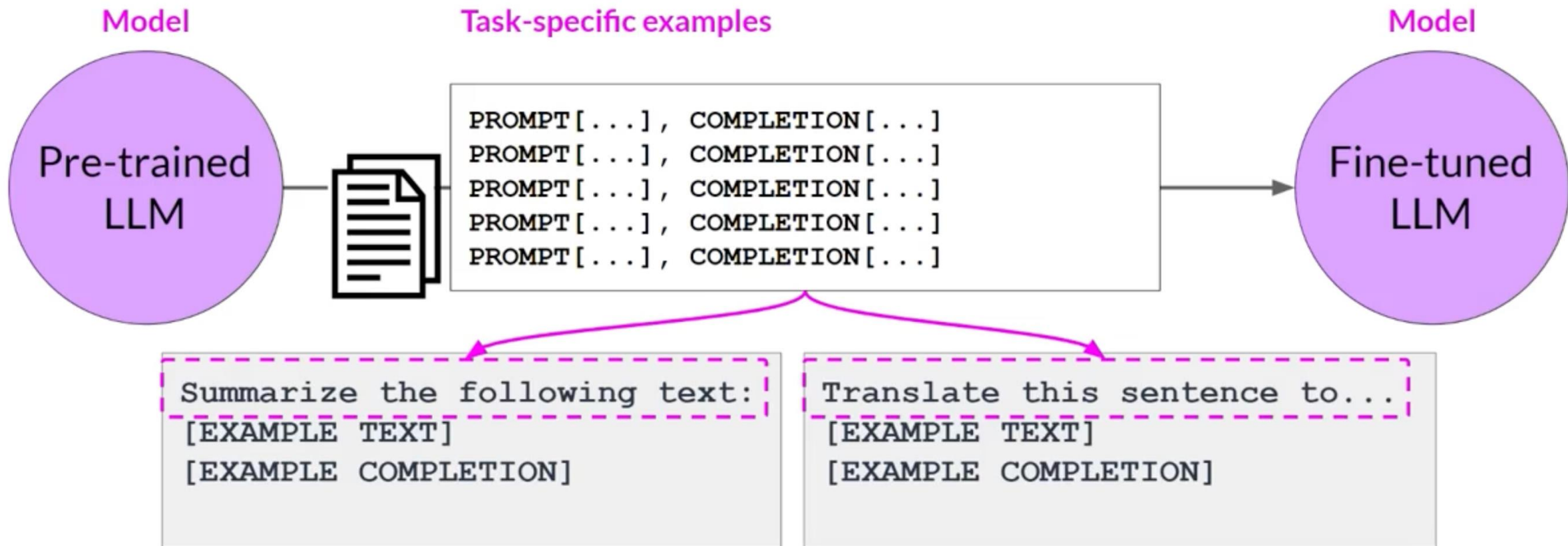
LLM fine-tuning





Using prompts to fine-tune LLMs with instruction

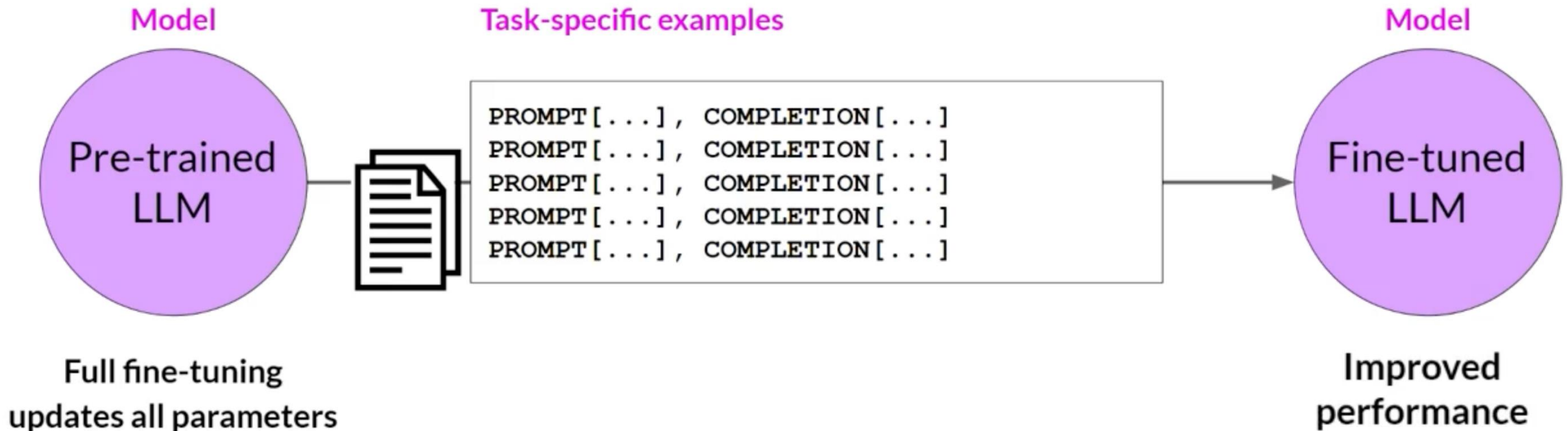
LLM fine-tuning





Using prompts to fine-tune LLMs with instruction

LLM fine-tuning



Sample prompt instruction templates

Classification / sentiment analysis

```
jinja: "Given the following review:\n{{review_body}}\n\npredict the associated rating\nfrom the following choices (1 being lowest and 5 being highest)\n- {{ answer_choices\n| join('\n- ') }}\n\n|||\n\n{{answer_choices[star_rating-1]}}"
```

Text generation

```
jinja: Generate a {{star_rating}}-star review (1 being lowest and 5 being highest)\nabout this product {{product_title}}. ||| {{review_body}}
```

Text summarization

```
jinja: "Give a short sentence describing the following product review:\n{{review_body}}\n\n|||\n\n{{review_headline}}"
```

LLM fine-tuning process

LLM fine-tuning

Prepared instruction dataset



Training splits

```
PROMPT [...], COMPLETION [...]  
PROMPT [...], COMPLETION [...]  
PROMPT [...], COMPLETION [...]  
PROMPT [...], COMPLETION [...]  
PROMPT [...], COMPLETION [...]
```

Training

```
PROMPT [...], COMPLETION [...]  
...
```

Validation

```
PROMPT [...], COMPLETION [...]  
...
```

Test

LLM fine-tuning process

LLM fine-tuning

Prepared instruction dataset



Prompt:

Classify this review:
I loved this DVD!

Sentiment:

Model

Pre-trained
LLM

LLM completion:

Classify this review:
I loved this DVD!

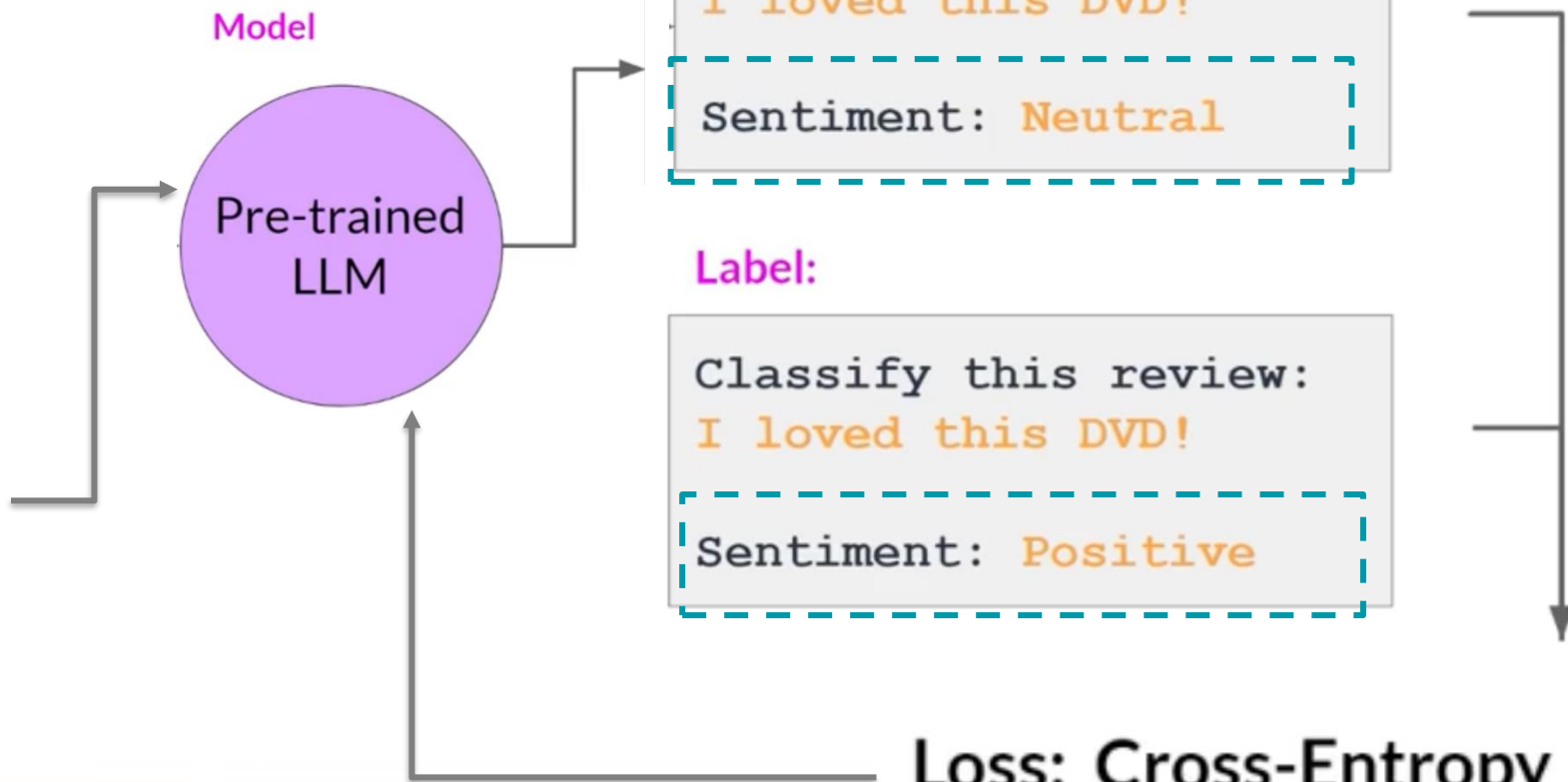
Sentiment: Neutral

Label:

Classify this review:
I loved this DVD!

Sentiment: Positive

Loss: Cross-Entropy





LLM fine-tuning process

LLM fine-tuning

Prepared instruction dataset



Training splits

```
PROMPT [...], COMPLETION [...]  
PROMPT [...], COMPLETION [...]  
PROMPT [...], COMPLETION [...]  
PROMPT [...], COMPLETION [...]  
PROMPT [...], COMPLETION [...]
```

Training

```
PROMPT [...], COMPLETION [...]  
...
```

Validation

validation_accuracy

```
PROMPT [...], COMPLETION [...]  
...
```

Test



LLM fine-tuning process

LLM fine-tuning

Prepared instruction dataset



Training splits

```
PROMPT[...], COMPLETION[...]  
PROMPT[...], COMPLETION[...]  
PROMPT[...], COMPLETION[...]  
PROMPT[...], COMPLETION[...]  
PROMPT[...], COMPLETION[...]
```

Training

```
PROMPT[...], COMPLETION[...]  
...
```

Validation

```
PROMPT[...], COMPLETION[...]  
...
```

Test

test_accuracy

LLM fine-tuning process



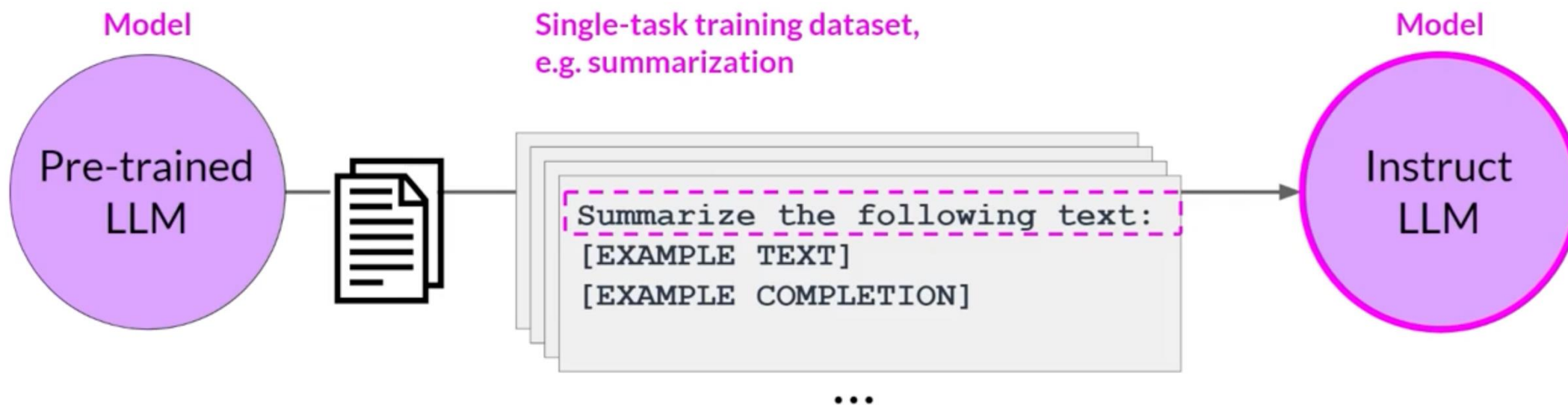


LLM fine-tuning process



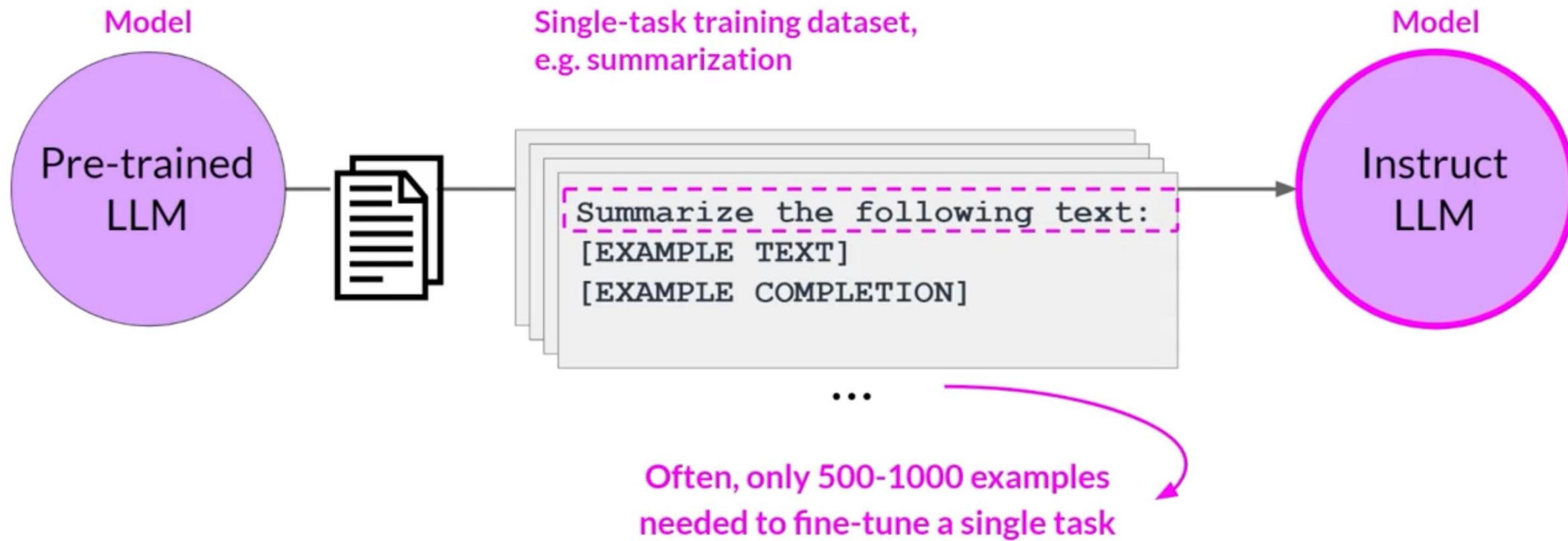


Fine-tuning on a single task





Fine-tuning on a single task





Catastrophic forgetting

- Fine-tuning can significantly increase the performance of a model on a specific task...

After fine-tuning

Prompt

```
Classify this review:  
I loved this DVD!  
Sentiment:
```

Model

LLM

Completion

```
Classify this review:  
I loved this DVD!  
Sentiment: POSITIVE
```



Catastrophic forgetting

- ...but can lead to reduction in ability on other tasks

After fine-tuning

Prompt

What is the name of
the cat?
Charlie the cat roamed
the garden at night.



Model



Completion

What is the name of
the cat?
Charlie the cat roamed
the garden at night.
The garden was
positive.



How to avoid Catastrophic forgetting?

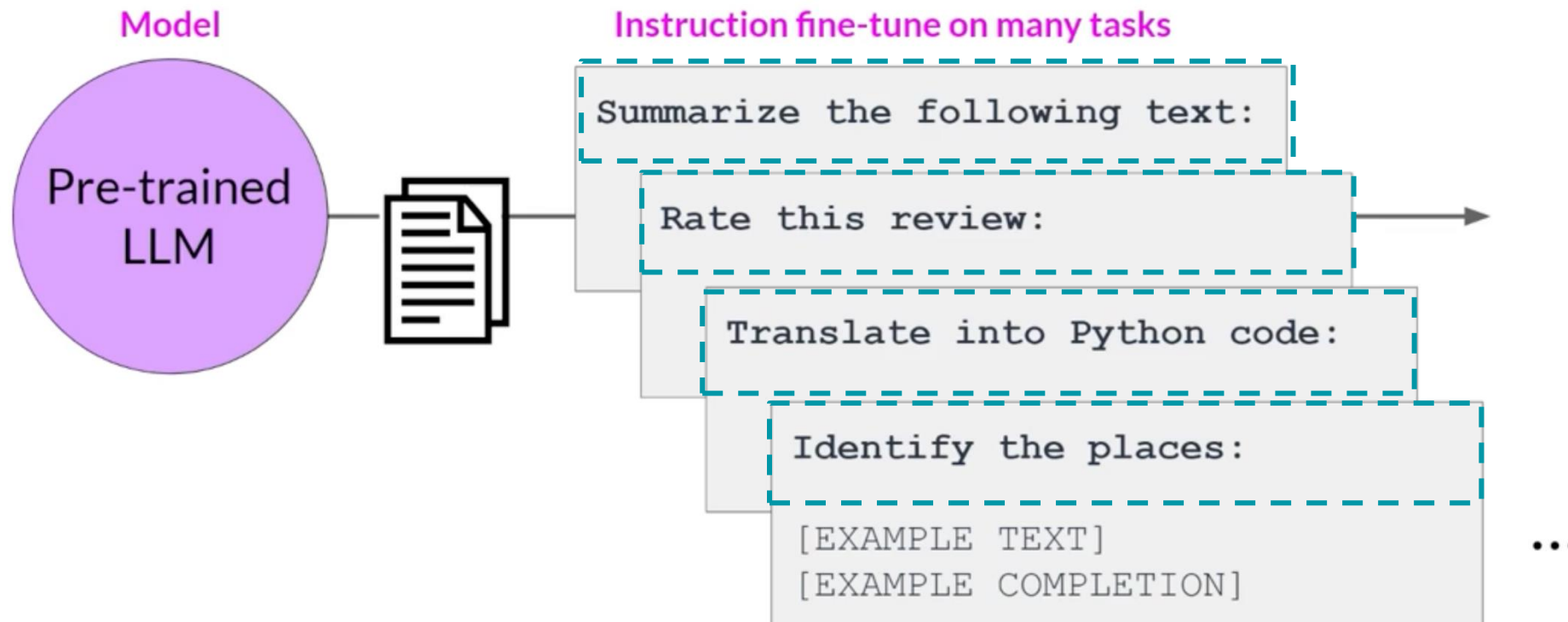
- First note that you may not have to!
- Fine tune on **multiple-tasks** at the same time.
- Consider **Parameter Efficient Fine Tuning (PEFT)**

Multi-task, Instruction Finetuning



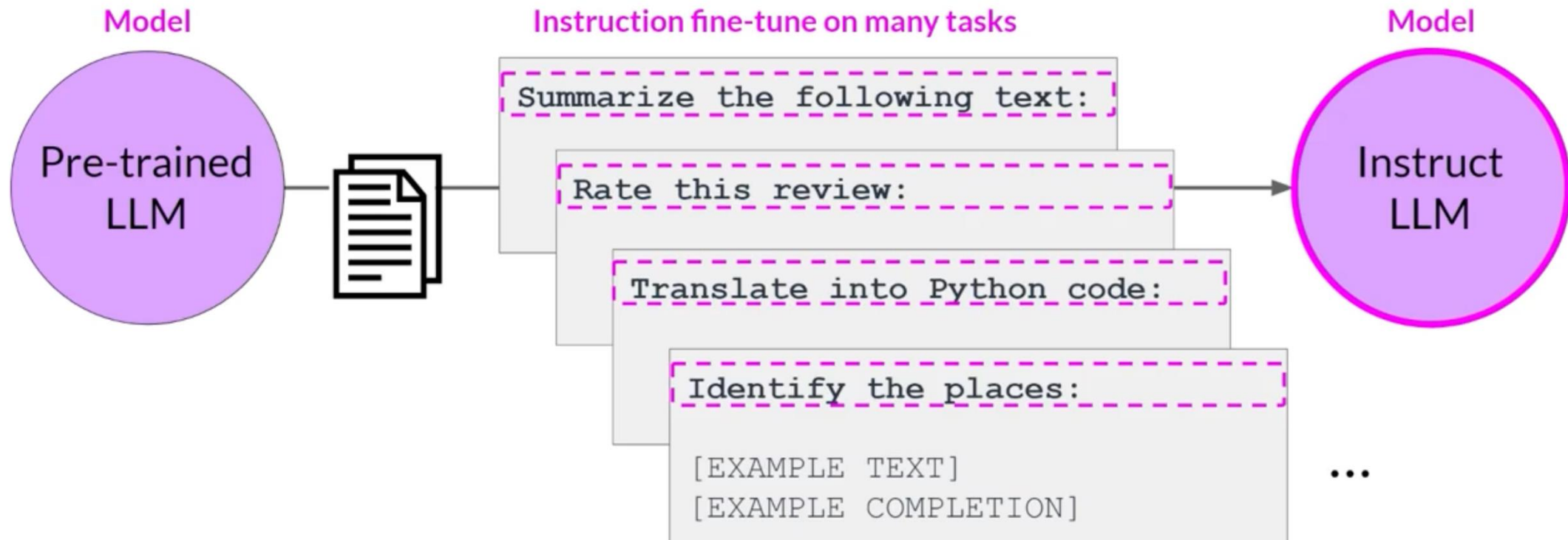


Multi-task, instruction fine-tuning



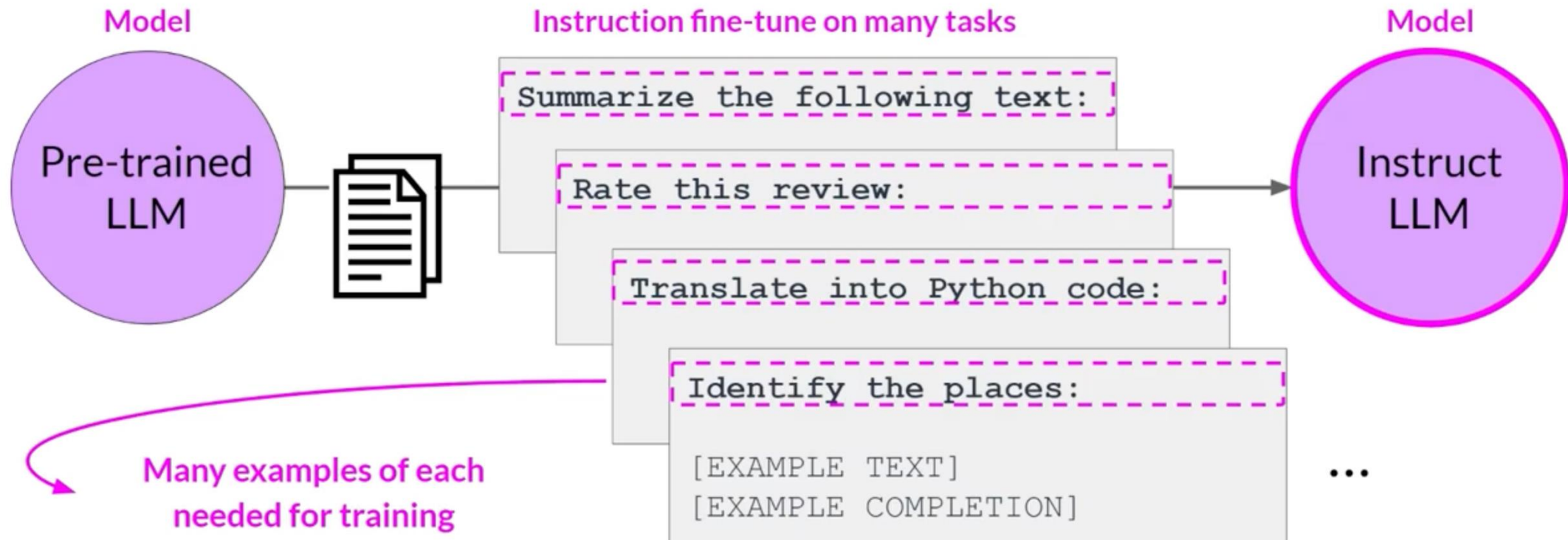


Multi-task, instruction fine-tuning





Multi-task, instruction fine-tuning





Instruction fine-tuning with FLAN

- FLAN models refer to a specific set of instructions used to perform instruction fine-tuning



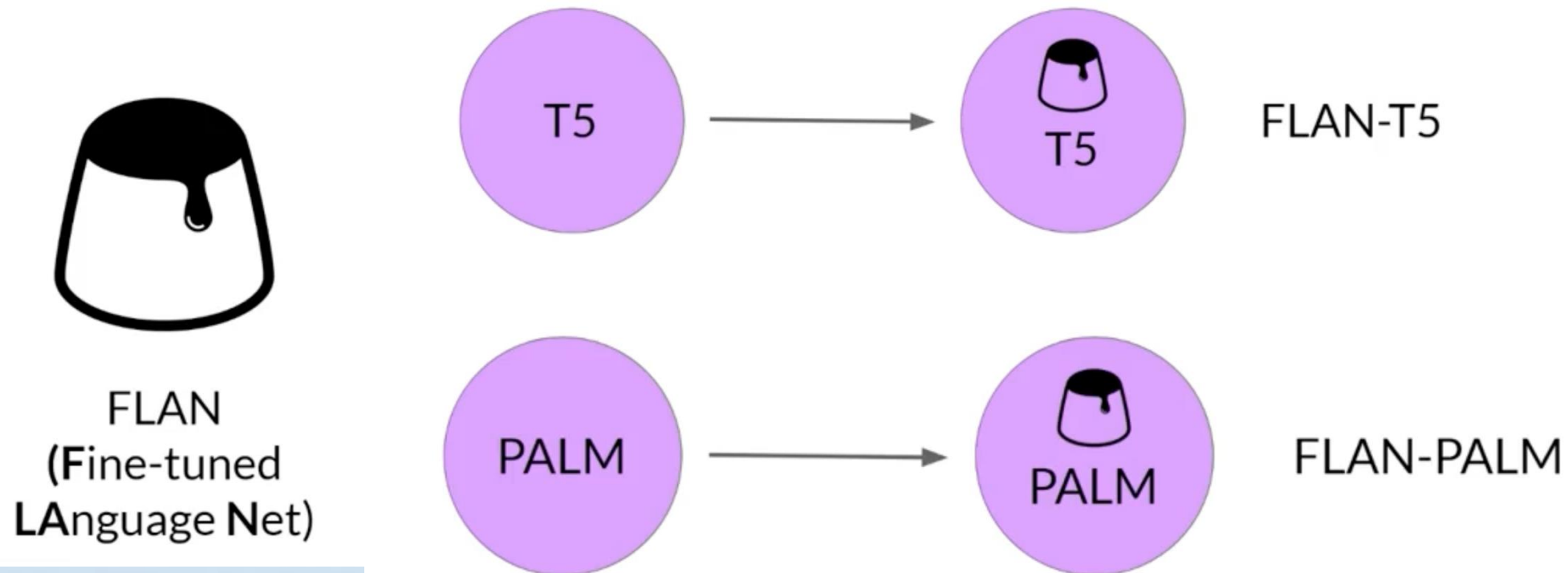
FLAN

“The metaphorical dessert to the main course of pretraining”



Instruction fine-tuning with FLAN

- FLAN models refer to a specific set of instructions used to perform instruction fine-tuning



FLAN-T5: Fine-tuned version of pre-trained T5 model

- FLAN-T5 is a great, general purpose, instruct model

FLAN-T5: Fine-tuned version of pre-trained T5 model

- FLAN-T5 is a great, general purpose, instruct model

T0-SF

- Commonsense Reasoning,
- Question Generation,
- Closed-book QA,
- Adversarial QA,
- Extractive QA

...

55 Datasets
14 Categories
193 Tasks

Muffin

- Natural language inference,
- Code instruction gen,
- Code repair
- Dialog context generation,
- Summarization (SAMSum)

...

69 Datasets
27 Categories
80 Tasks

CoT (reasoning)

- Arithmetic reasoning,
- Commonsense reasoning
- Explanation generation,
- Sentence composition,
- Implicit reasoning,

...

9 Datasets
1 Category
9 Tasks

Natural Instructions

- Cause effect classification,
- Commonsense reasoning,
- Named Entity Recognition,
- Toxic Language Detection,
- Question answering

...

372 Datasets
108 Categories
1554 Tasks

Source: Chung et al. 2022, "Scaling Instruction-Finetuned Language Models"

SAMSum: A dialogue dataset

Sample prompt training dataset (**samsum**) to fine-tune FLAN-T5 from pretrained T5

 Datasets: samsum		Tasks:  Summarization	Languages:  English
dialogue (string)		summary (string)	
"Amanda: I baked cookies. Do you want some? Jerry: Sure! Amanda: I'll bring you tomorrow :-)"		"Amanda baked cookies and will bring Jerry some tomorrow."	
"Olivia: Who are you voting for in this election? Oliver: Liberals as always. Olivia: Me too!! Oliver: Great"		"Olivia and Olivier are voting for liberals in this election. "	
"Tim: Hi, what's up? Kim: Bad mood tbh, I was going to do lots of stuff but ended up procrastinating Tim: What did...		"Kim may try the pomodoro technique recommended by Tim to get more stuff done."	

Source: <https://huggingface.co/datasets/samsum>, <https://github.com/google-research/FLAN/blob/2c79a31/flan/v2/templates.py#L3285>

Sample FLAN-T5 prompt templates

```
"samsun": [  
  ("{"dialogue}"\n\nBriefly summarize that dialogue.", "{"summary}"),  
  ("Here is a dialogue:\n{dialogue}\n\nWrite a short summary!",  
   "{"summary}"),  
  ("Dialogue:\n{dialogue}\n\nWhat is a summary of this dialogue?",  
   "{"summary}"),  
  ("{"dialogue}"\n\nWhat was that dialogue about, in two sentences or less?",  
   "{"summary}"),  
  ("Here is a dialogue:\n{dialogue}\n\nWhat were they talking about?",  
   "{"summary}"),  
  ("Dialogue:\n{dialogue}\n\nWhat were the main points in that "  
   "conversation?", "{"summary}"),  
  ("Dialogue:\n{dialogue}\n\nWhat was going on in that conversation?",  
   "{"summary}"),  
]
```

Improving FLAN-T5's summarization capabilities

 ShopBot

Hi there! How can I help you today?

 I need to return a pair of jeans that I purchased

Of course, I'd be happy to help you. When did you buy your item?

 Two weeks ago.

Goal: Summarize conversations to identify actions to take

Improving FLAN-T5's summarization capabilities

Further fine-tune FLAN-T5 with a domain-specific instruction dataset (dialogsum)

Datasets:

knkarthick

/

dialogsum

like

13

Tasks:

Summarization

Text2Text Generation

Text Generation

Languages:

English

Multilinguality:

monolingual

Size Categories:

Language Creators:

expert-generated

Annotations Creators:

expert-generated

Source Datasets:

original

License:

mit

Dataset card

Files and versions

Community 4

Dataset Preview

Split

train (12.5k rows)

id (string)	dialogue (string)	summary (string)
"train_0"	"#Person1#: Hi, Mr. Smith. I'm Doctor Hawkins. Why are you here today? #Person2#: I found it would be a good..."	"Mr. Smith's getting a check-up, and Doctor Hawkins advises him to have one every year. Hawkins'll give some..."
"train_1"	"#Person1#: Hello Mrs. Parker, how have you been? #Person2#: Hello Dr. Peters. Just fine thank you. Ricky..."	"Mrs Parker takes Ricky for his vaccines. Dr. Peters checks the record and then gives Ricky a vaccine."
"train_2"	"#Person1#: Excuse me, did you see a set of keys? #Person2#: What kind of keys? #Person1#: Five keys and a small foot ornament. #Person2#: What a shame! I didn't see them. #Person1#: Well, can you help me look for it? That's my first time here. #Person2#: Sure. It's my pleasure. I'd like to help you look for the missing keys. #Person1#: It's very kind of you. #Person2#: It's not a big deal. Hey, I found them. #Person1#: Oh, thank God! I don't know how to thank you, guys. #Person2#: You're welcome."	"#Person1#'s looking for a set of keys and asks for #Person2#'s help to find them."

Example support-dialog summarization

Prompt (created from template)

Summarize the following conversation.

Tommy: Hello. My name is Tommy Sandals, I have a reservation.

Mike: May I see some identification, sir, please?

Tommy: Sure. Here you go.

Mike: Thank you so much. Have you got a credit card, Mr. Sandals?

Tommy: I sure do.

Mike: Thank you, sir. You'll be in room 507, nonsmoking, queen bed.

Tommy: That's great, thank you!

Mike: Enjoy your stay!

Summary **before** fine-tuning FLAN-T5 with our dataset

Prompt (created from template)

Summarize the following conversation.

Tommy: Hello. My name is Tommy Sandals, I have a reservation.

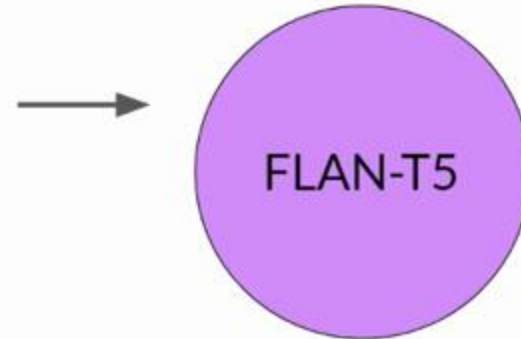
Mike: May I see some

...
...
...

Tommy: That's great, thank you!

Mike: Enjoy your stay!

Model



Completion (Summary)

Tommy Sandals has a reservation for a room at the Venetian Hotel in Las Vegas.

Adequate completion, but does not match human baseline.

Human baseline summary:
Tommy Sandals has got a reservation. Mike asks for his identification and credit card and helps his check-in.



Summary **after** fine-tuning FLAN-T5 with our dataset

Prompt (created from template)

Summarize the following conversation.

Tommy: Hello. My name is Tommy Sandals, I have a reservation.

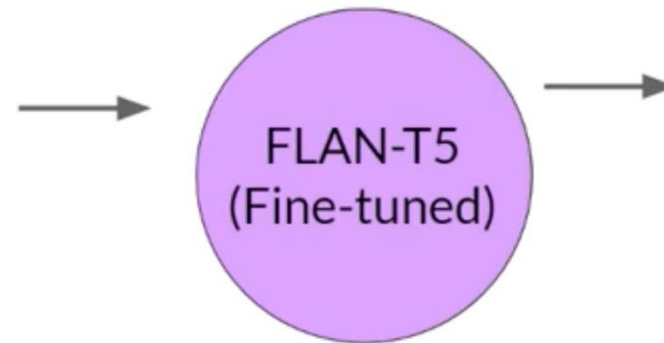
Mike: May I see some

...
...
...

Tommy: That's great, thank you!

Mike: Enjoy your stay!

Model



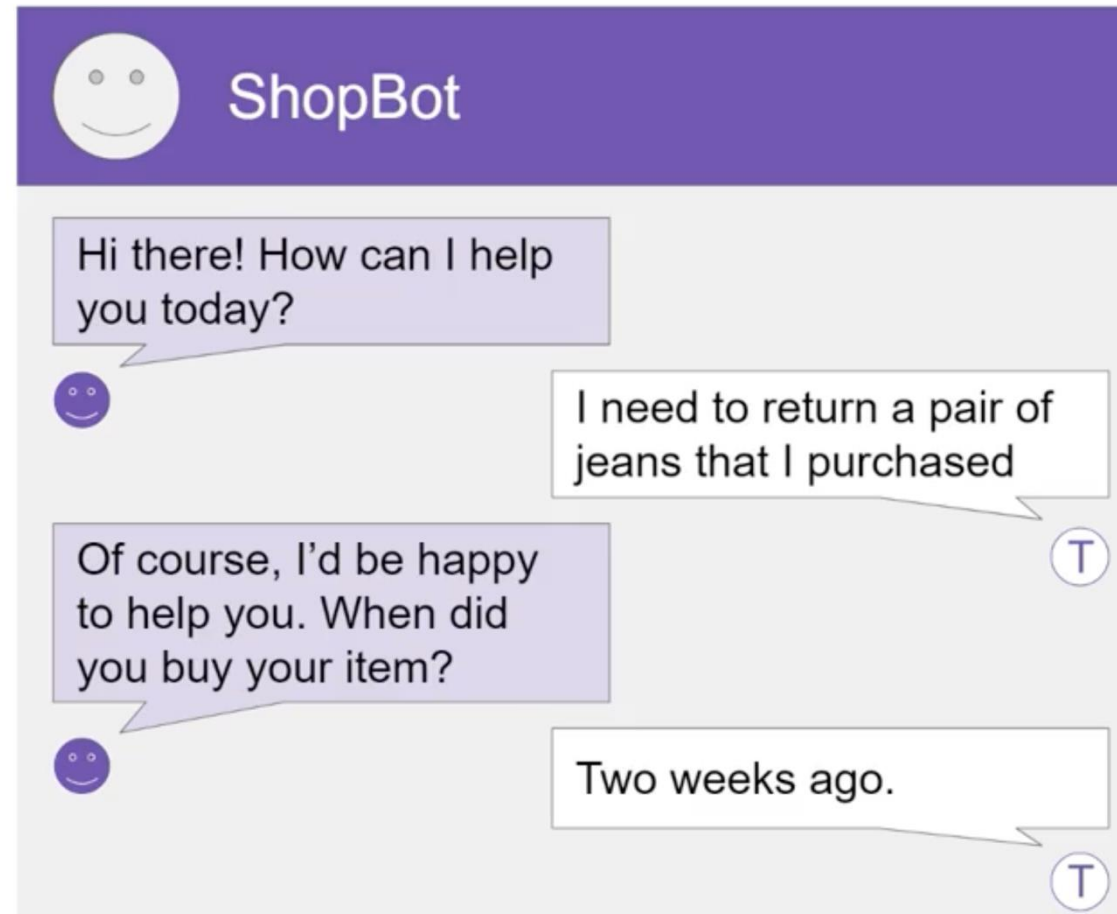
Completion (Summary)

Tommy Sandals has a reservation and checks in showing his ID and credit card. Mike helps him to check in and approves his reservation.

*Better summary,
more-closely matches
human baseline.*



Fine-tuning with your own data





Full fine-tuning of large LLMs is challenging





Full fine-tuning of large LLMs is challenging

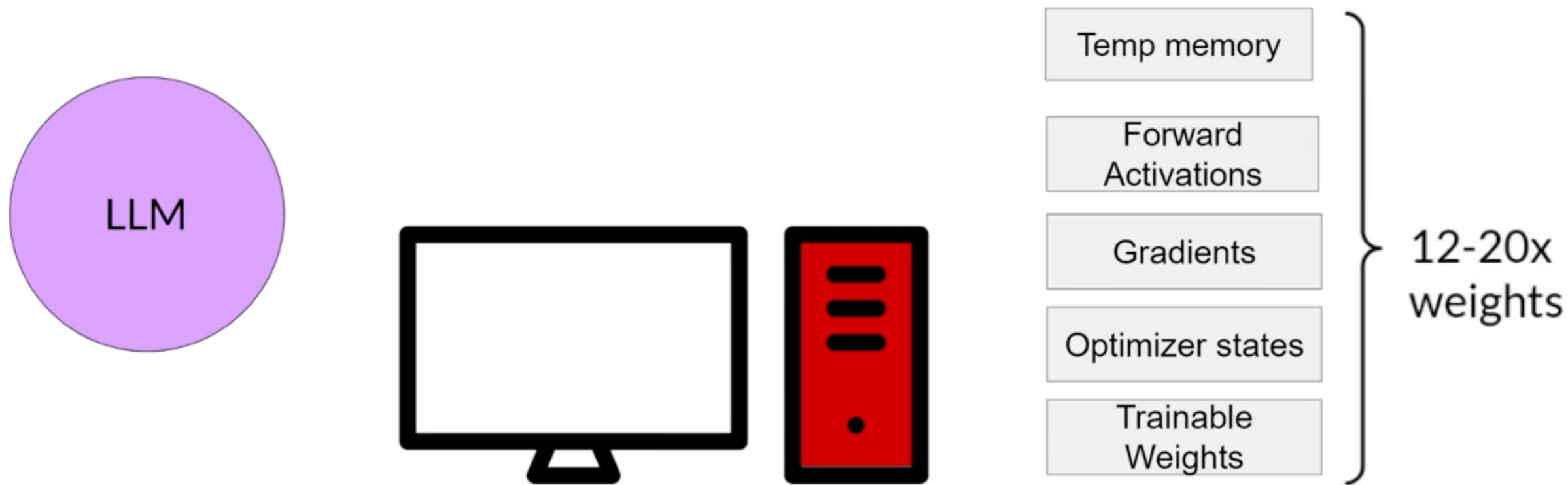


Optimizer states

Trainable
Weights

Full fine-tuning of large LLMs is challenging

ring 2026



Parameter efficient fine-tuning (PEFT)



Parameter efficient fine-tuning (PEFT)

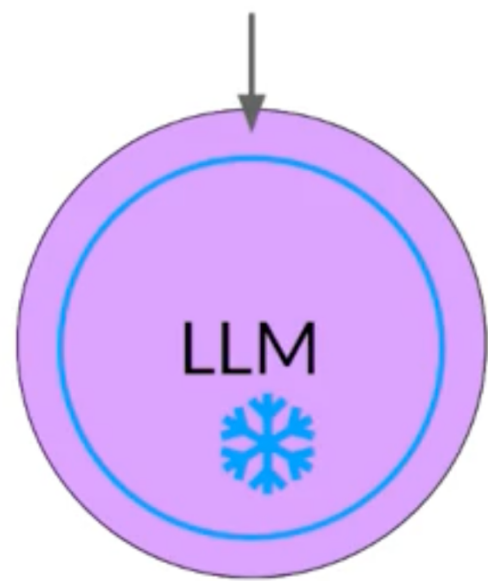


LLM with most layers
frozen



Parameter efficient fine-tuning (PEFT)

Small number of trainable layers



LLM with most layers frozen



Parameter efficient fine-tuning (PEFT)



Parameter efficient fine-tuning (PEFT)



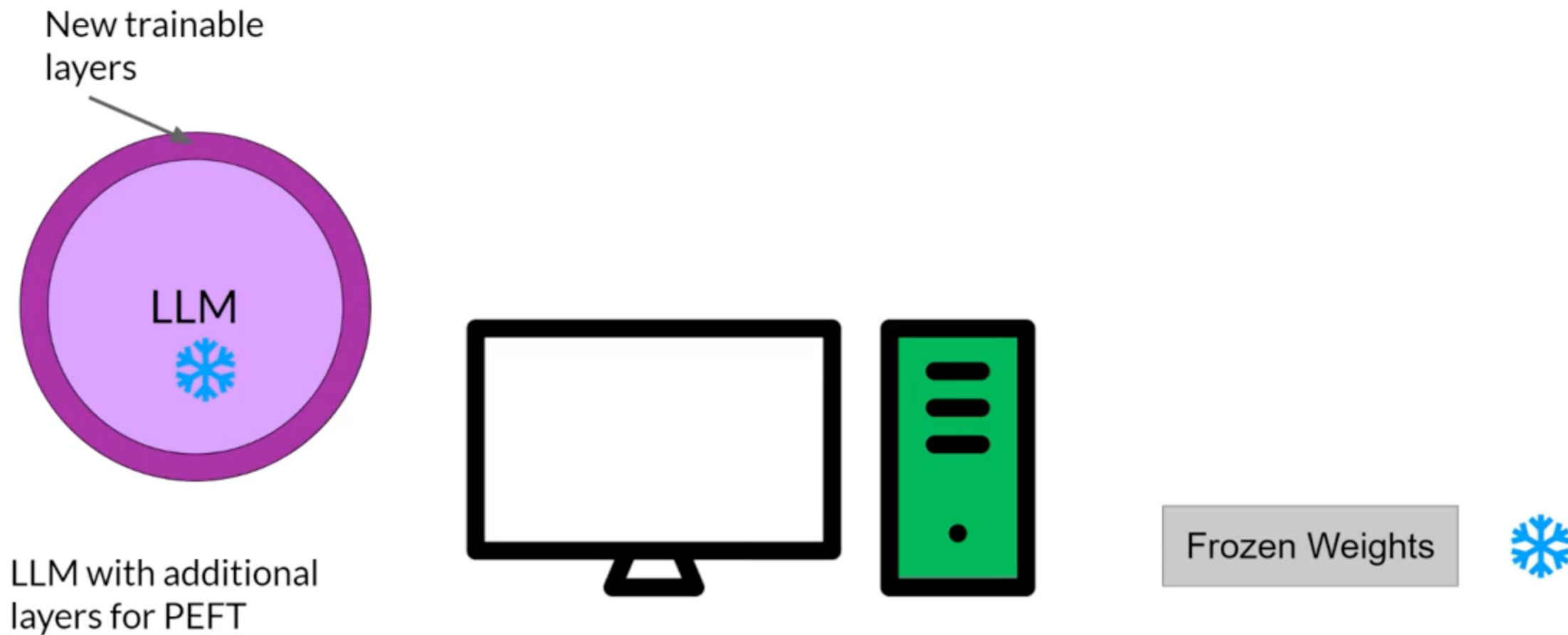
LLM with additional
layers for PEFT



Parameter efficient fine-tuning (PEFT)



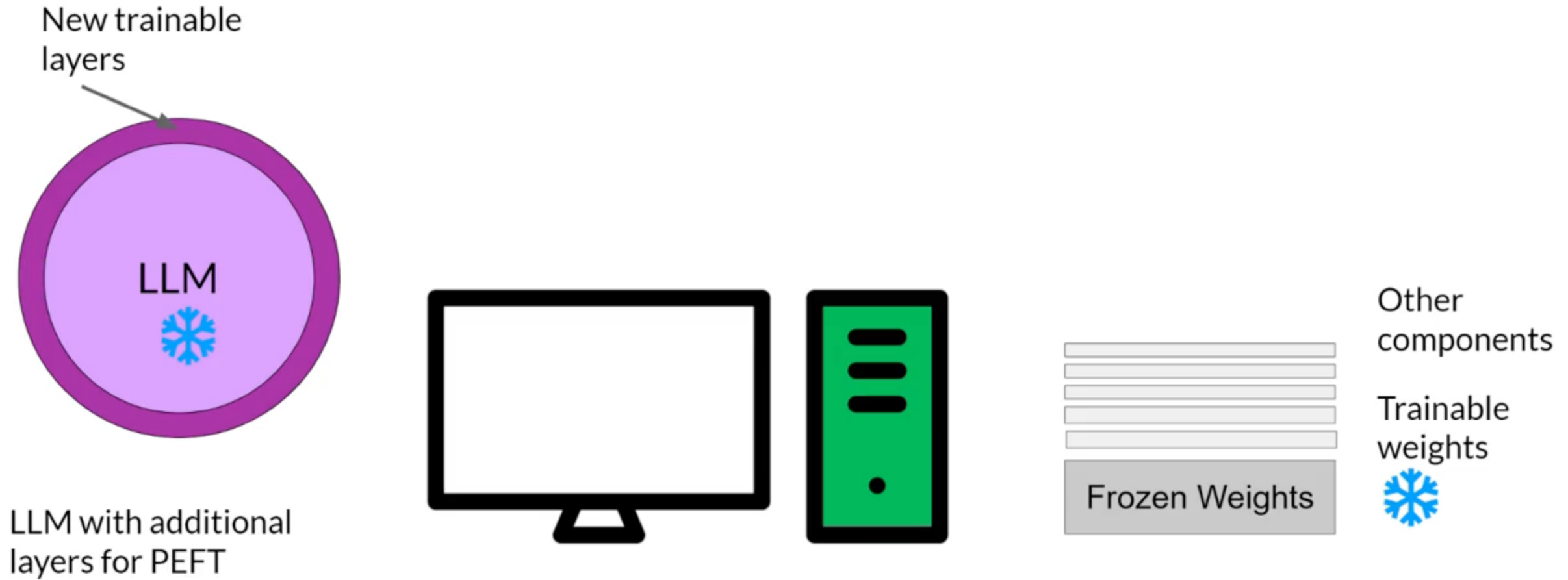
Parameter efficient fine-tuning (PEFT)



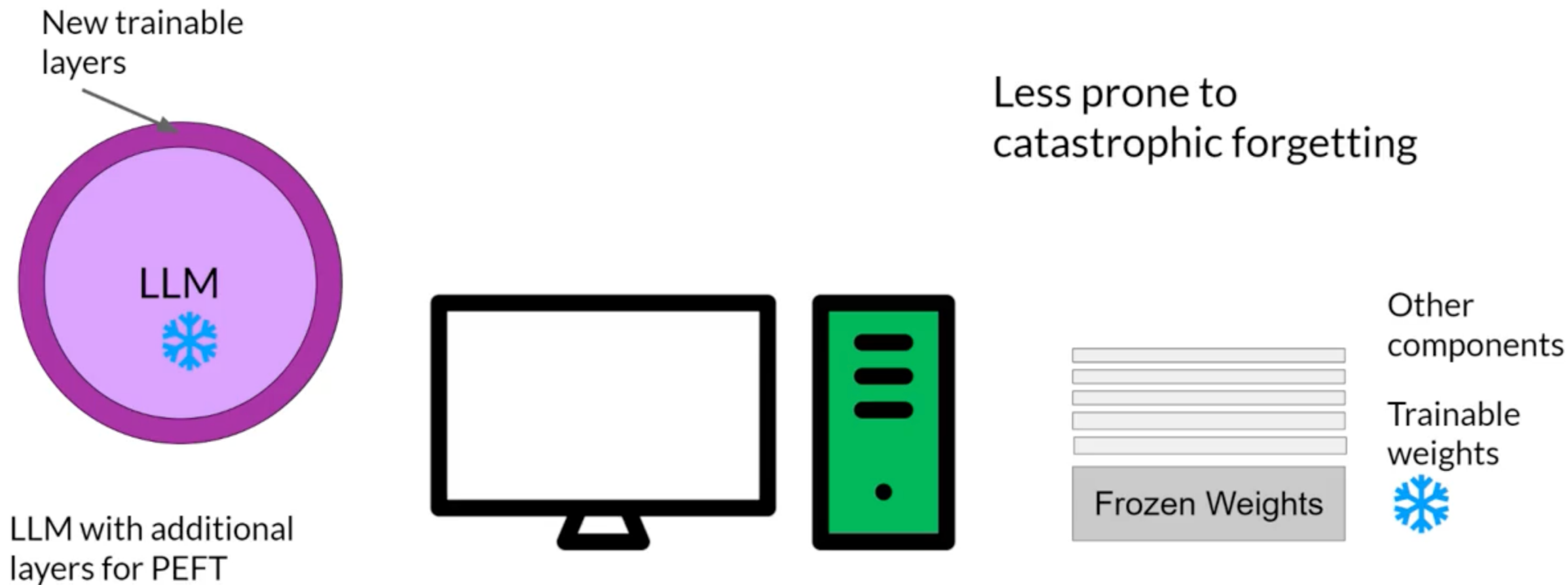
Parameter efficient fine-tuning (PEFT)



Parameter efficient fine-tuning (PEFT)



Parameter efficient fine-tuning (PEFT)





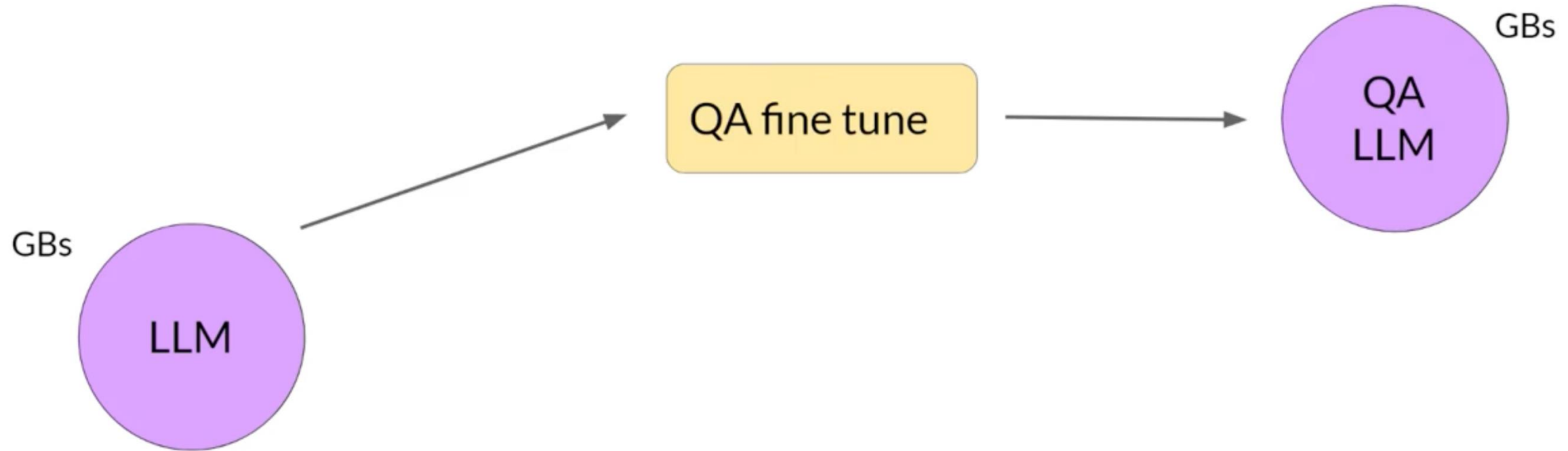
Full fine-tuning creates full copy of original LLM per task

GBs



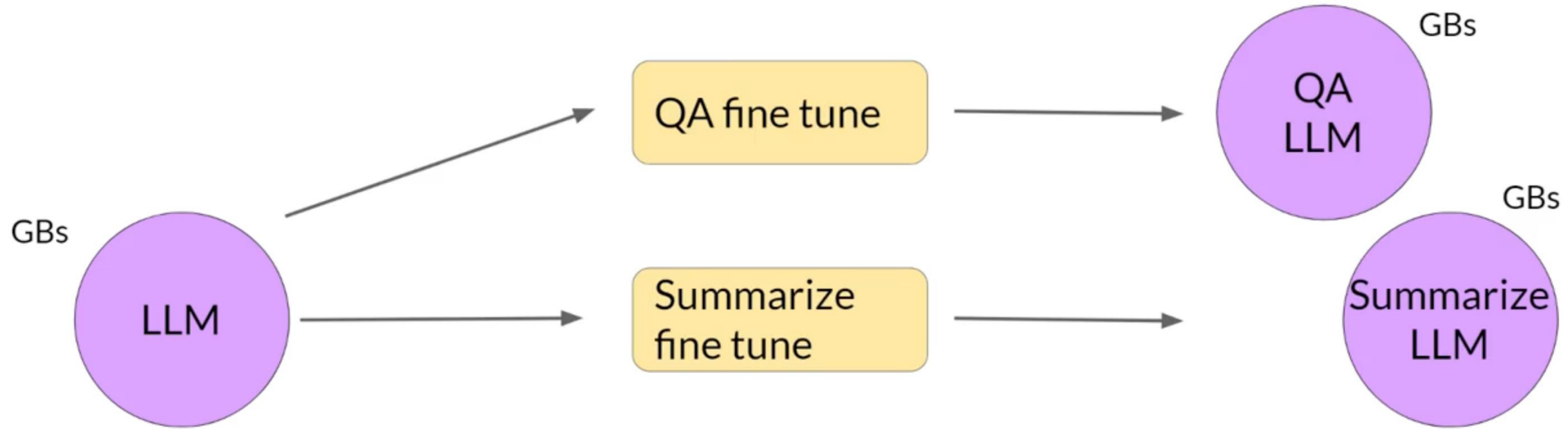


Full fine-tuning creates full copy of original LLM per task



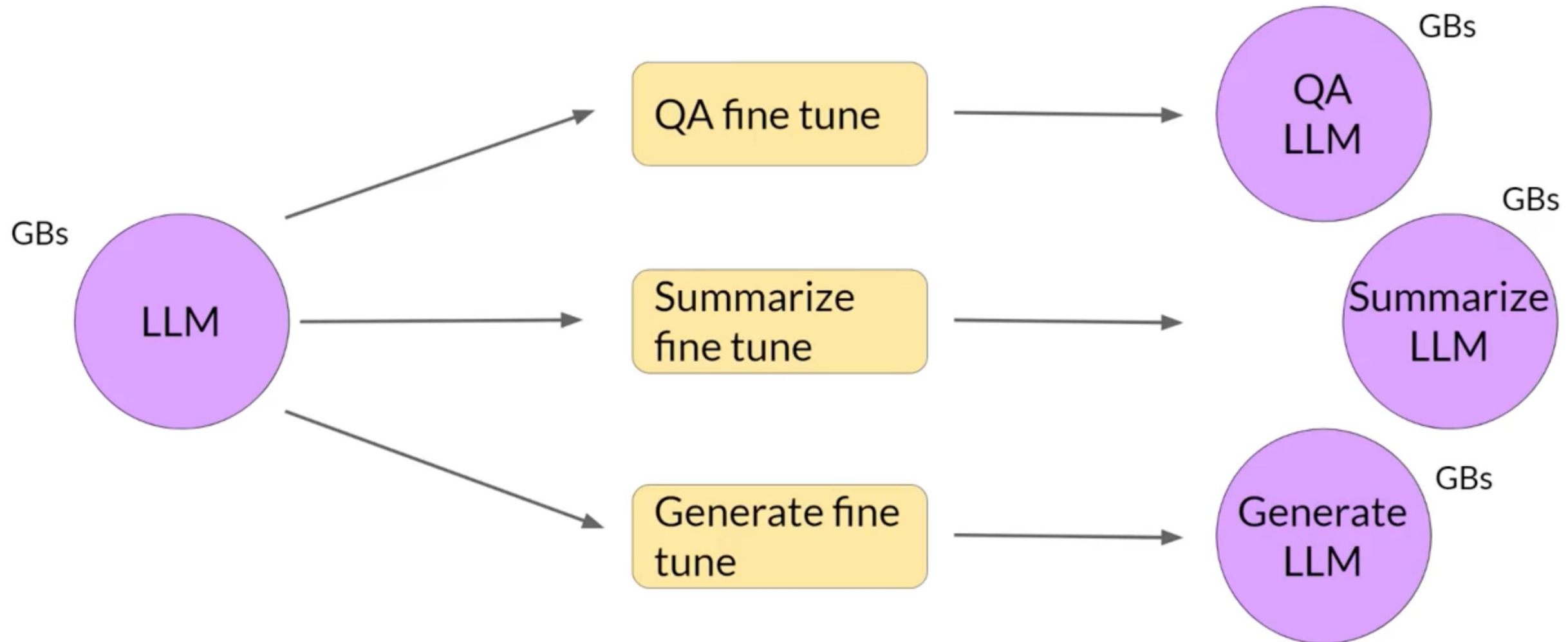


Full fine-tuning creates full copy of original LLM per task





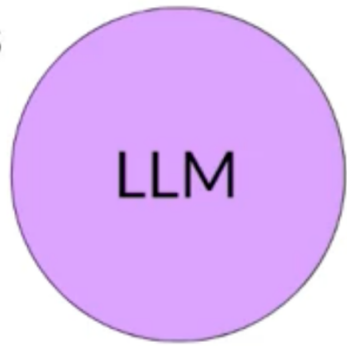
Full fine-tuning creates full copy of original LLM per task





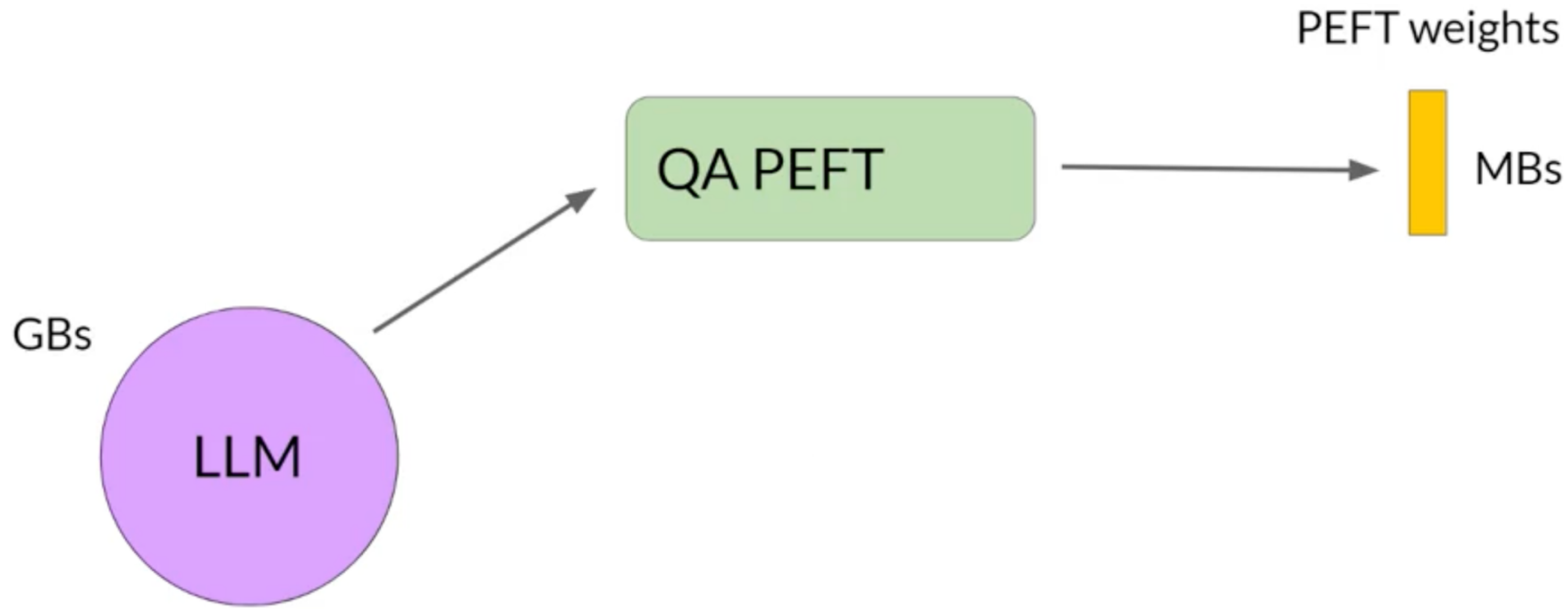
PEFT fine-tuning saves space and is flexible

GBs



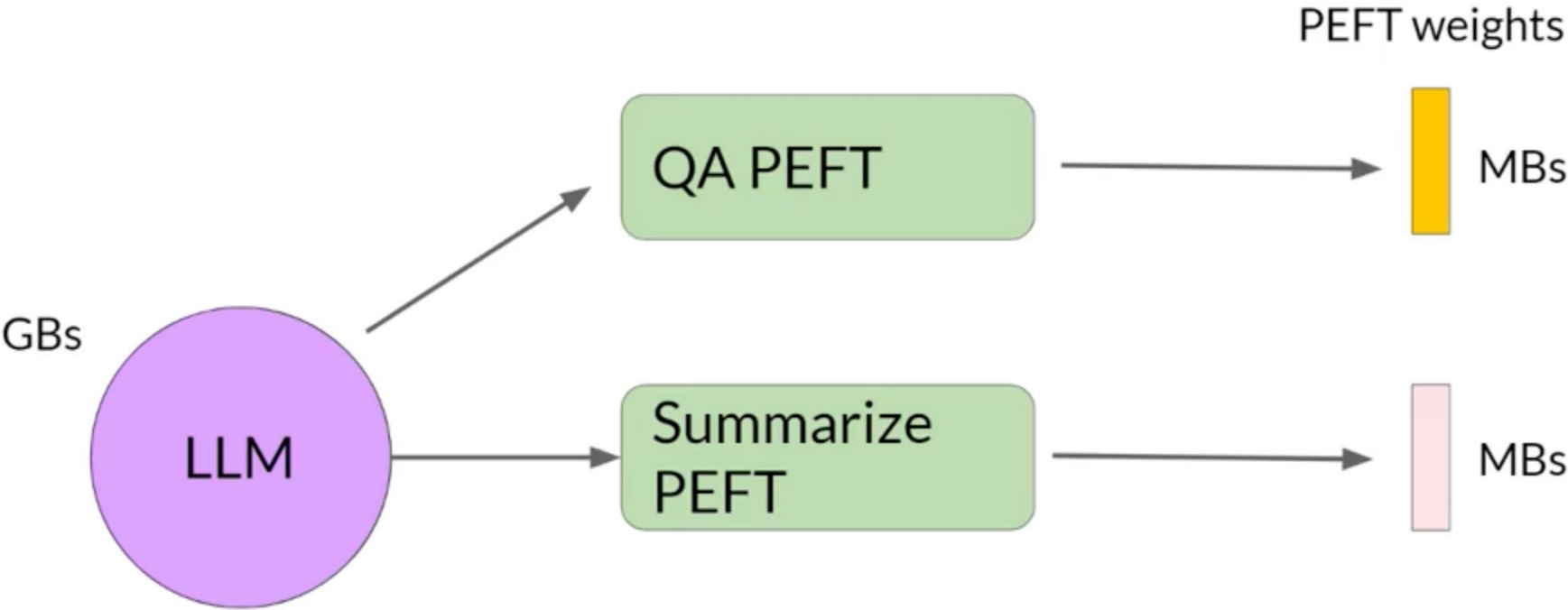


PEFT fine-tuning saves space and is flexible



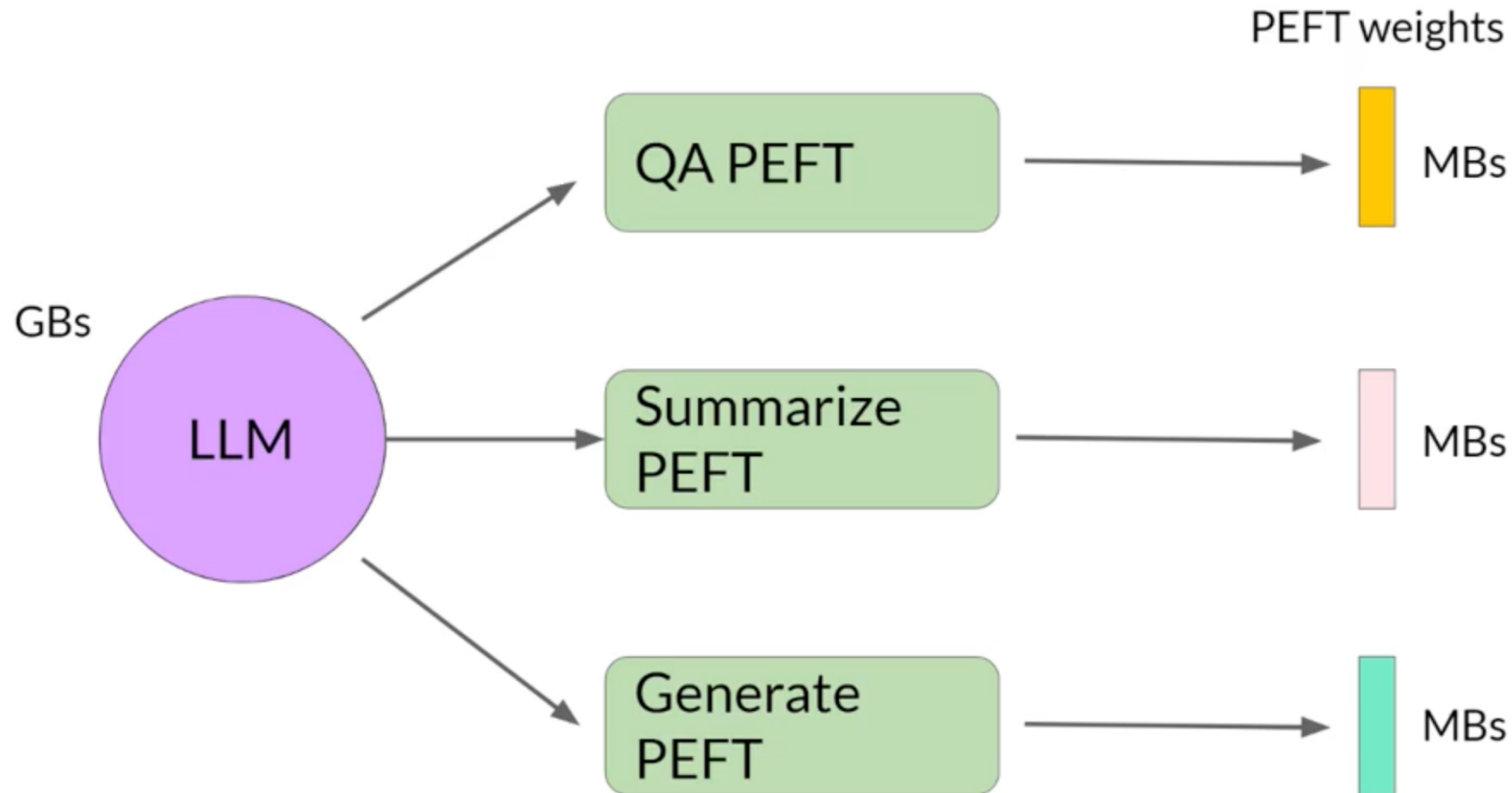


PEFT fine-tuning saves space and is flexible



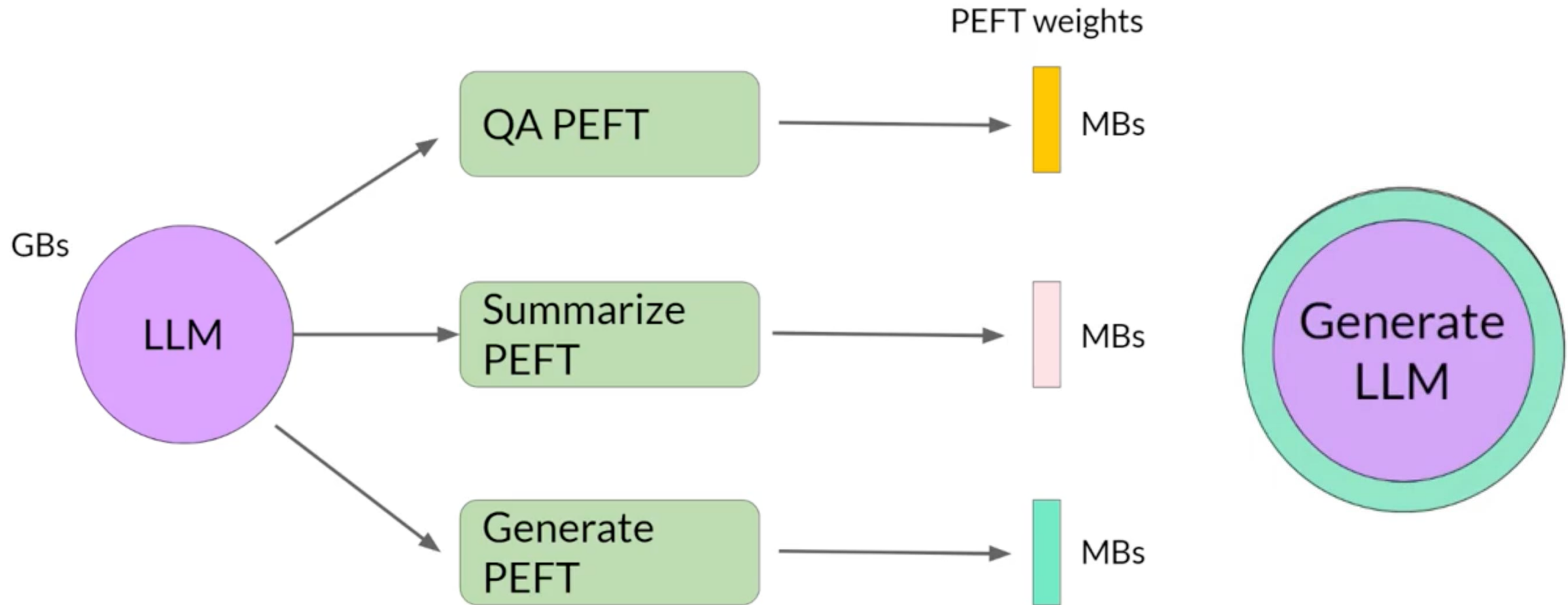


PEFT fine-tuning saves space and is flexible



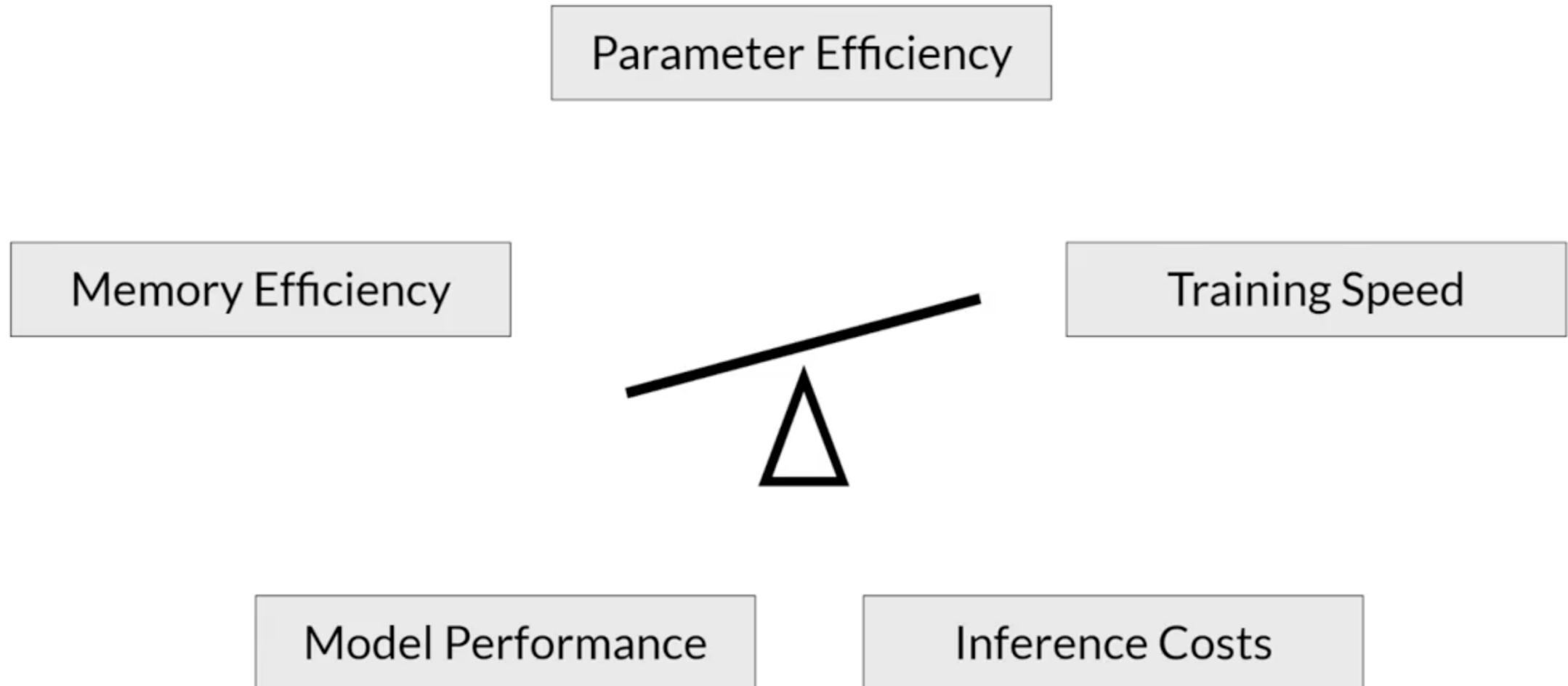


PEFT fine-tuning saves space and is flexible





PEFT Trade-offs





PEFT methods



PEFT methods

Selective

Select subset of initial LLM parameters to fine-tune

PEFT methods

Selective

Select subset of initial LLM parameters to fine-tune

Reparameterization

Reparameterize model weights using a low-rank representation

LoRA

Source: Lialin et al. 2023, "Scaling Down to Scale Up: A Guide to Parameter-Efficient Fine-Tuning",



PEFT methods

Selective

Select subset of initial LLM parameters to fine-tune

Reparameterization

Reparameterize model weights using a low-rank representation

LoRA

Additive

Add trainable layers or parameters to model

PEFT methods

Selective

Select subset of initial LLM parameters to fine-tune

Reparameterization

Reparameterize model weights using a low-rank representation

LoRA

Additive

Add trainable layers or parameters to model

Adapters



PEFT methods

Selective

Select subset of initial LLM parameters to fine-tune

Reparameterization

Reparameterize model weights using a low-rank representation

LoRA

Additive

Add trainable layers or parameters to model

Adapters

Soft Prompts



PEFT methods

Selective

Select subset of initial LLM parameters to fine-tune

Reparameterization

Reparameterize model weights using a low-rank representation

LoRA

Additive

Add trainable layers or parameters to model

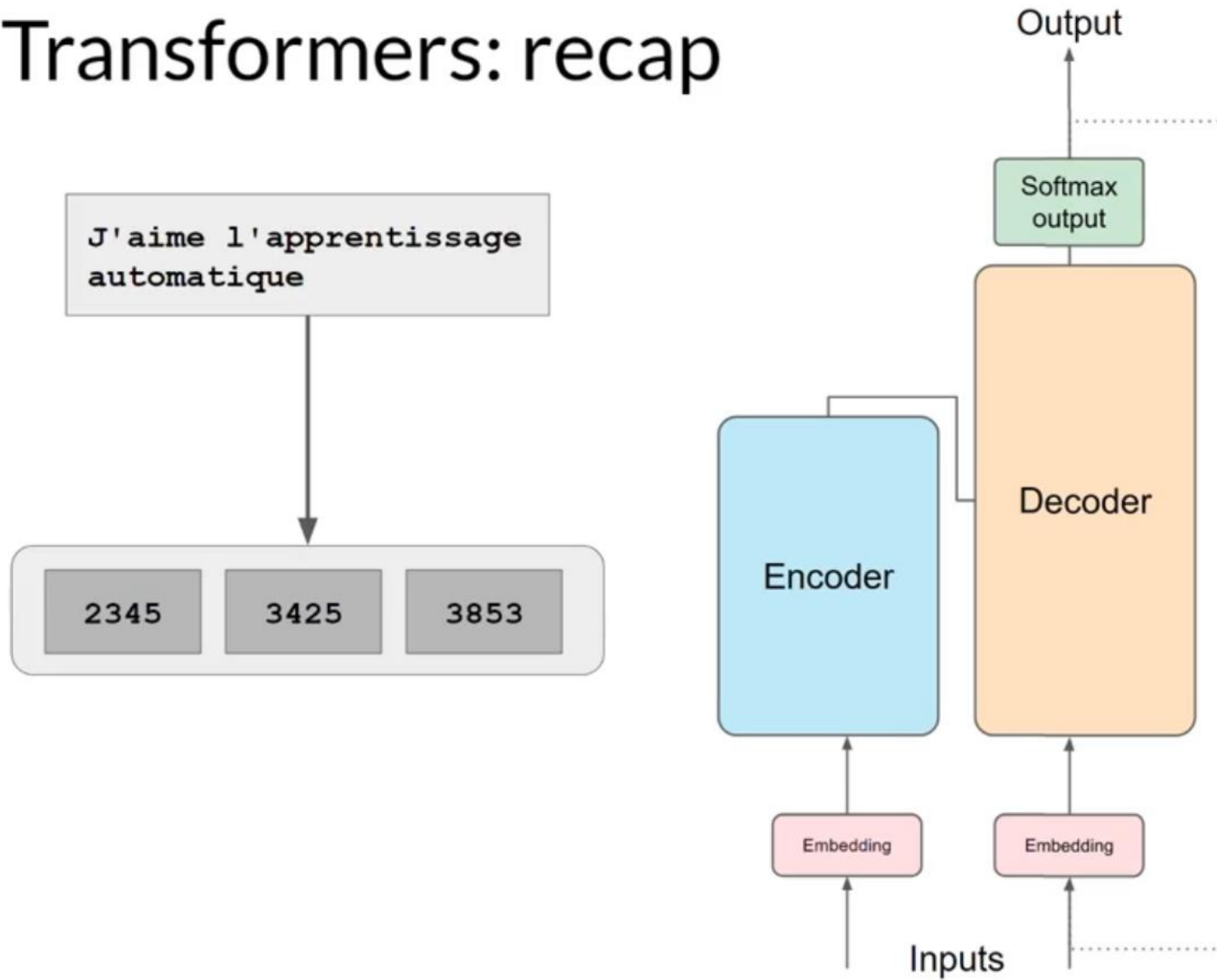
Adapters

Soft Prompts
Prompt Tuning

Low-Rank Adaptation of Large Language Models (LoRA)

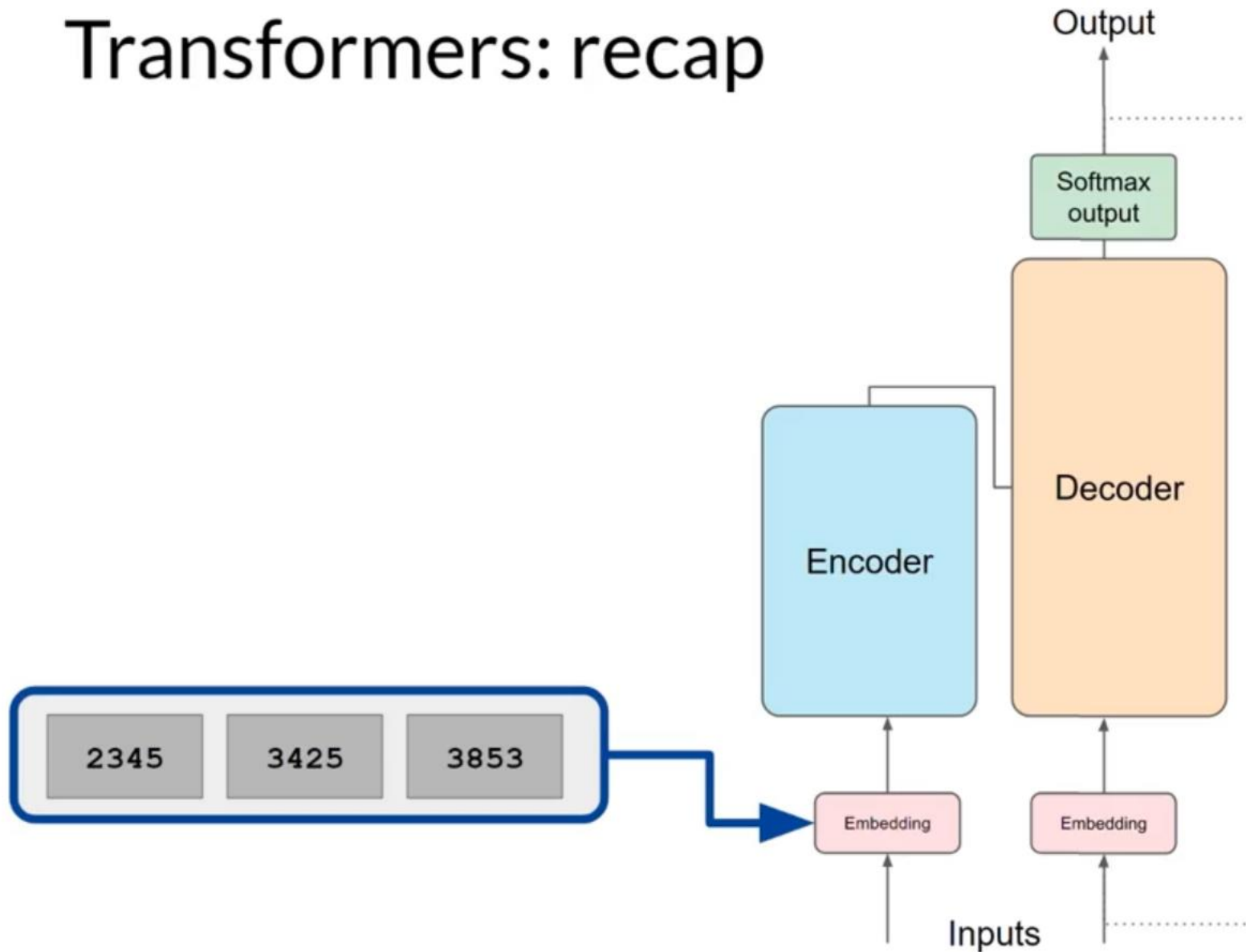


Transformers: recap



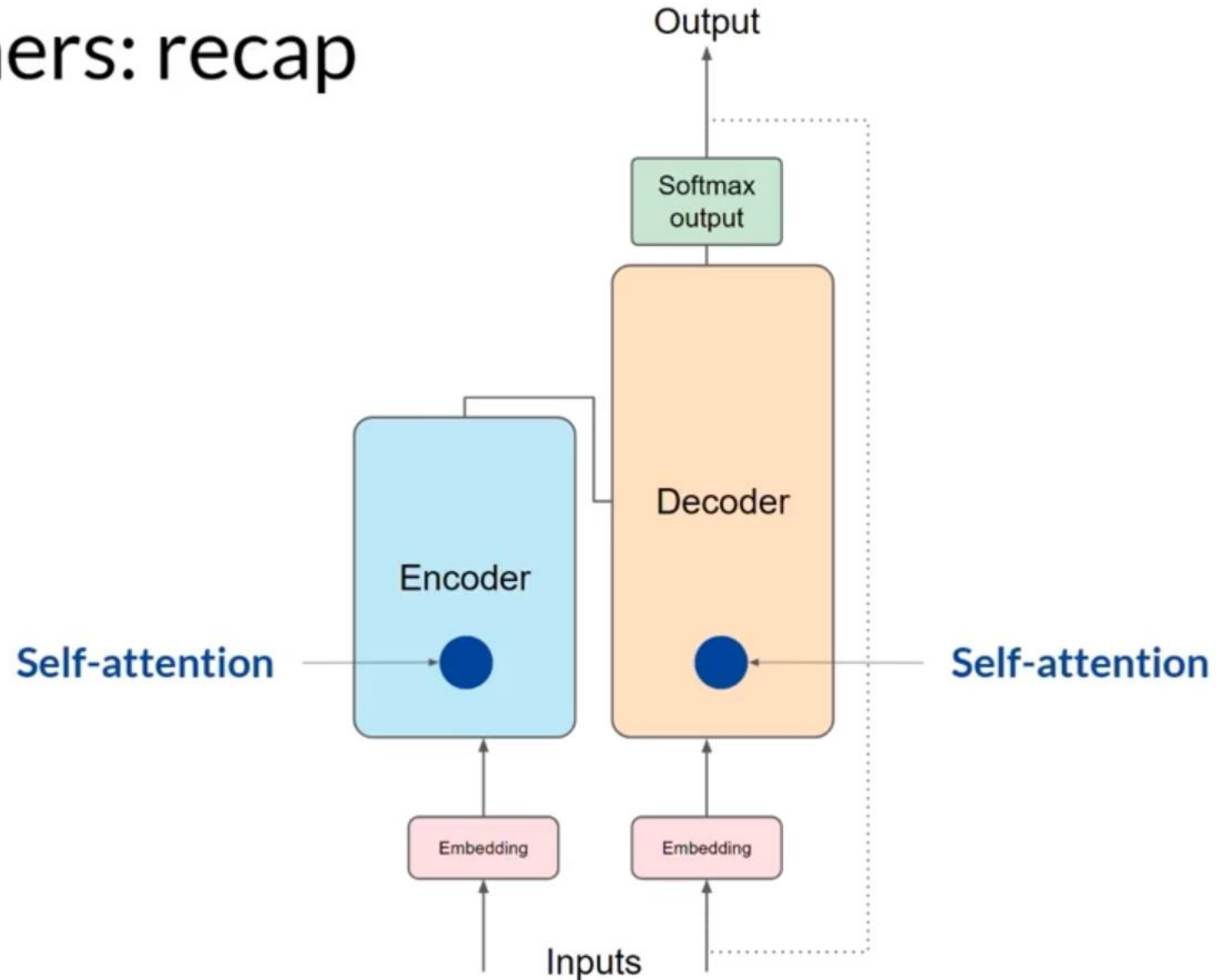


Transformers: recap



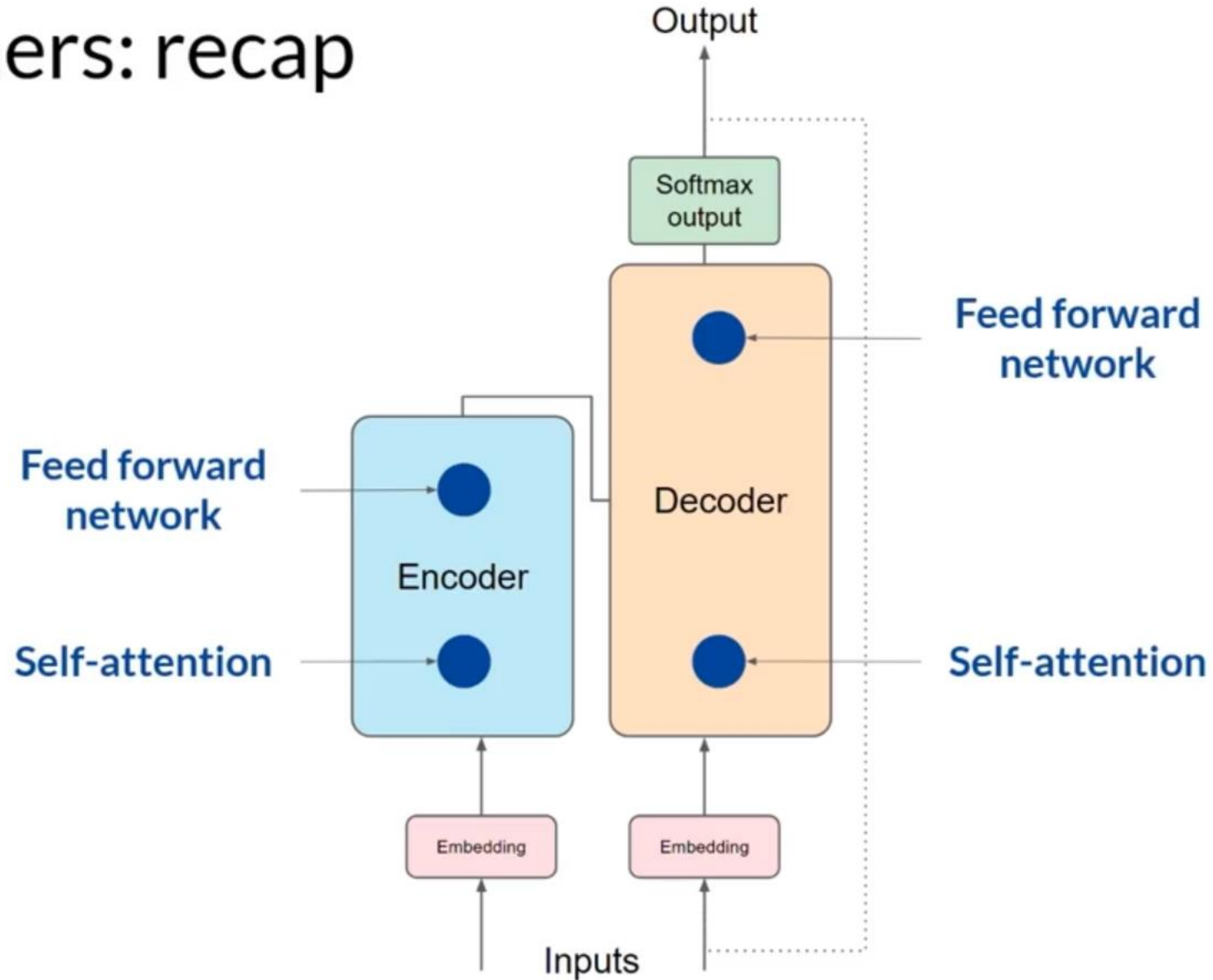


Transformers: recap



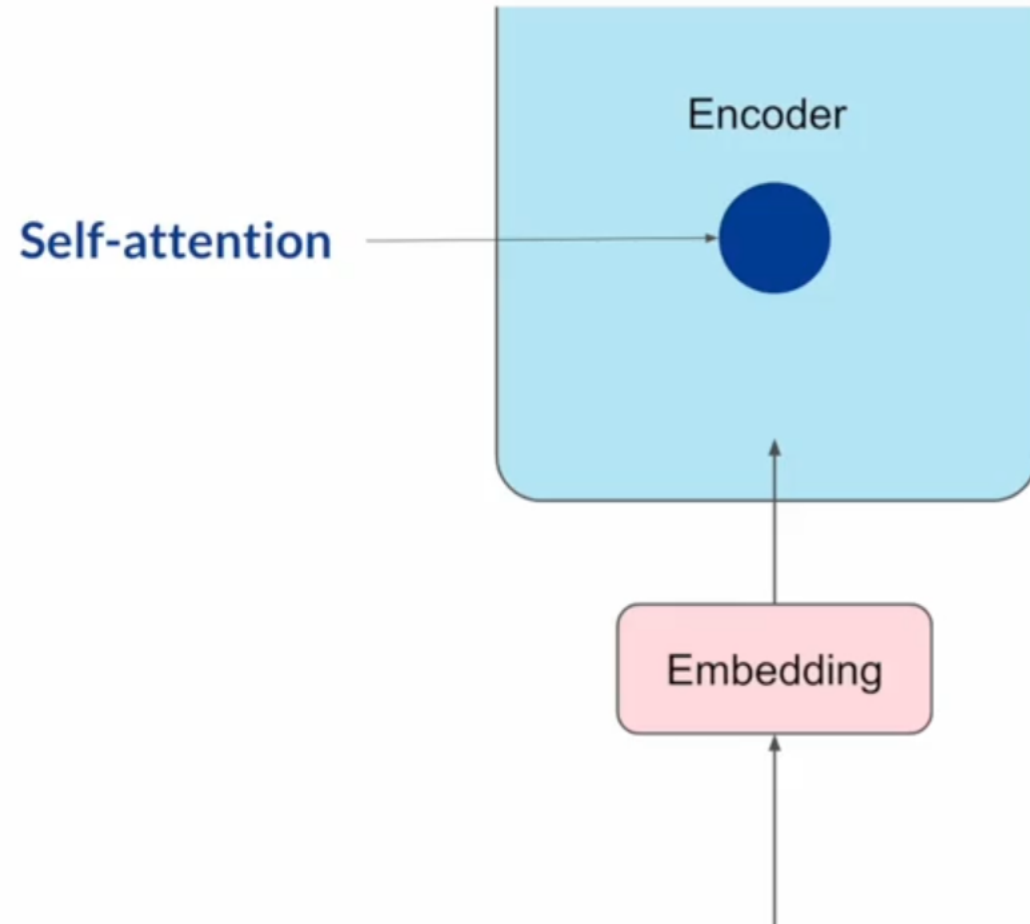


Transformers: recap



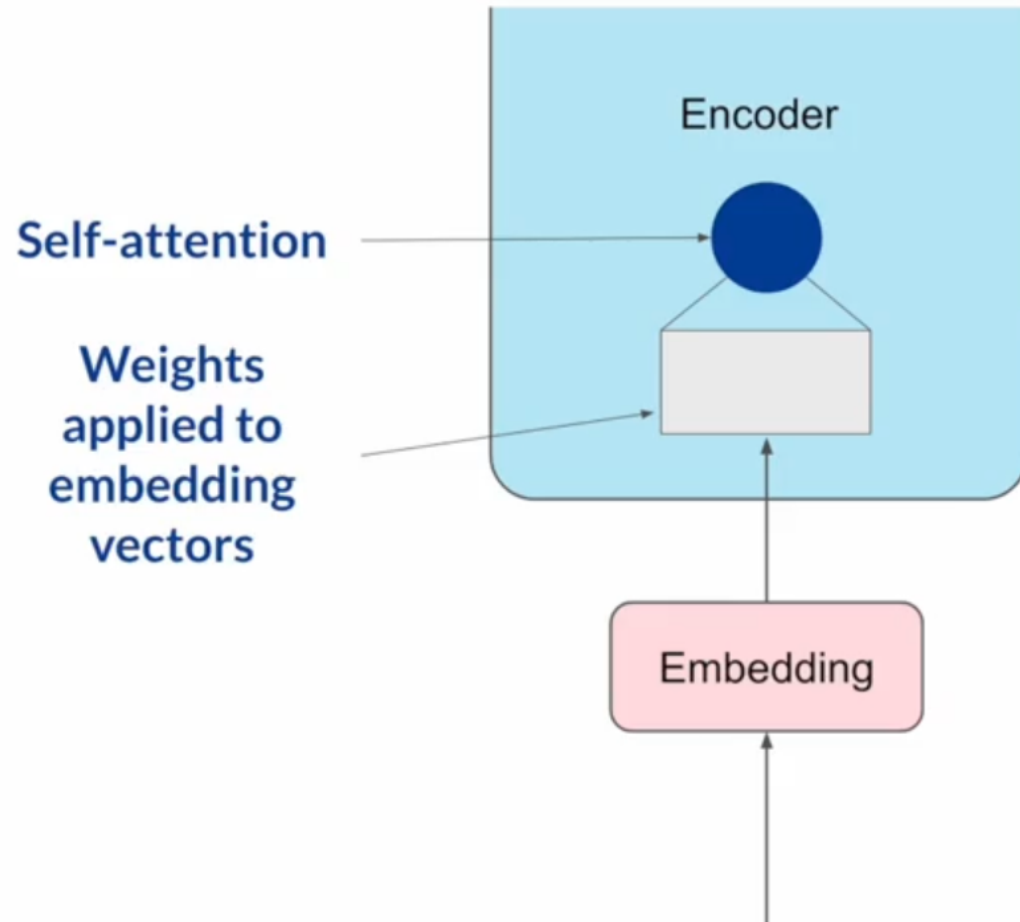


LoRA: Low Rank Adaption of LLMs



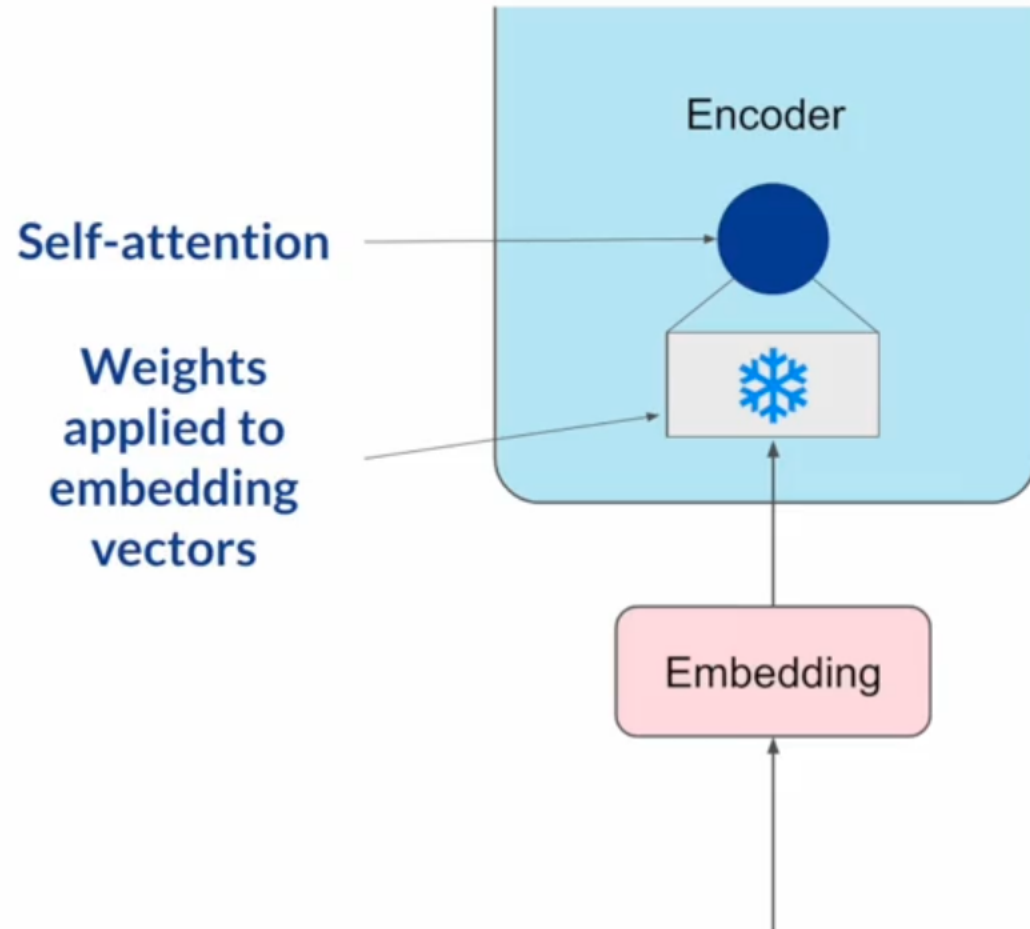


LoRA: Low Rank Adaption of LLMs





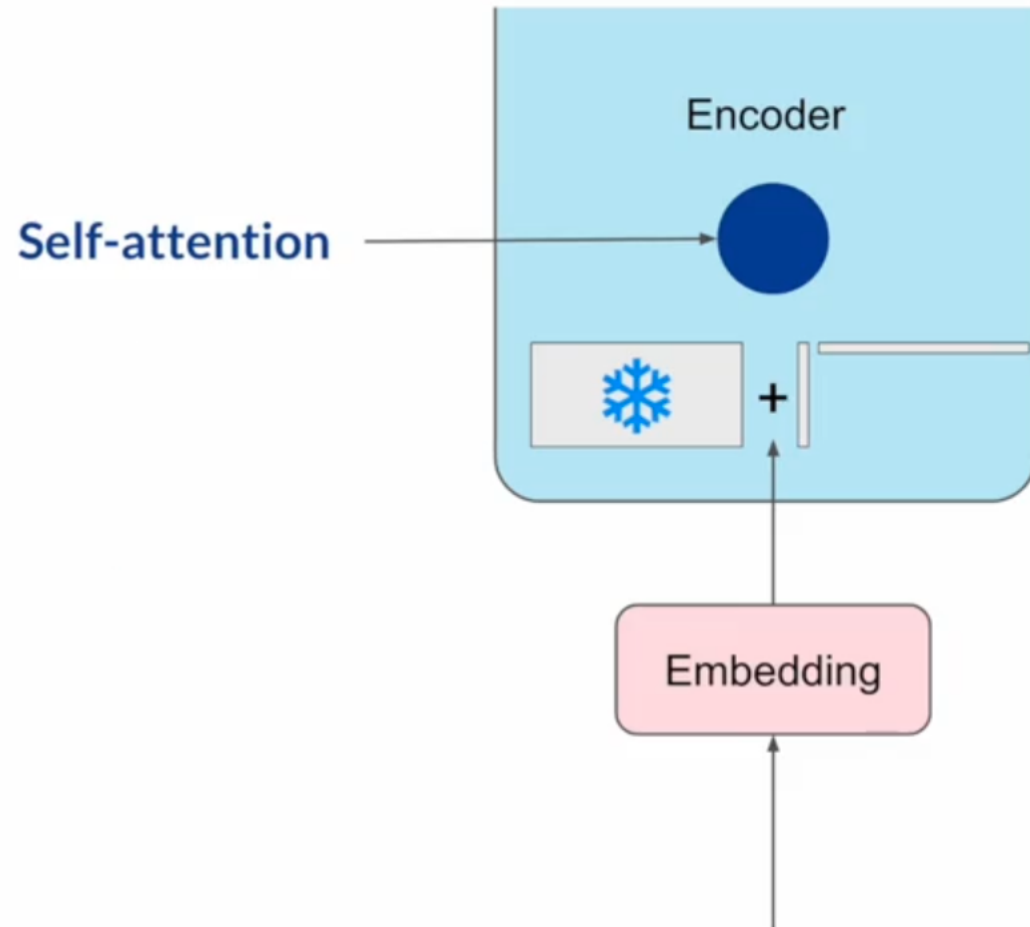
LoRA: Low Rank Adaption of LLMs



1. Freeze most of the original LLM weights.



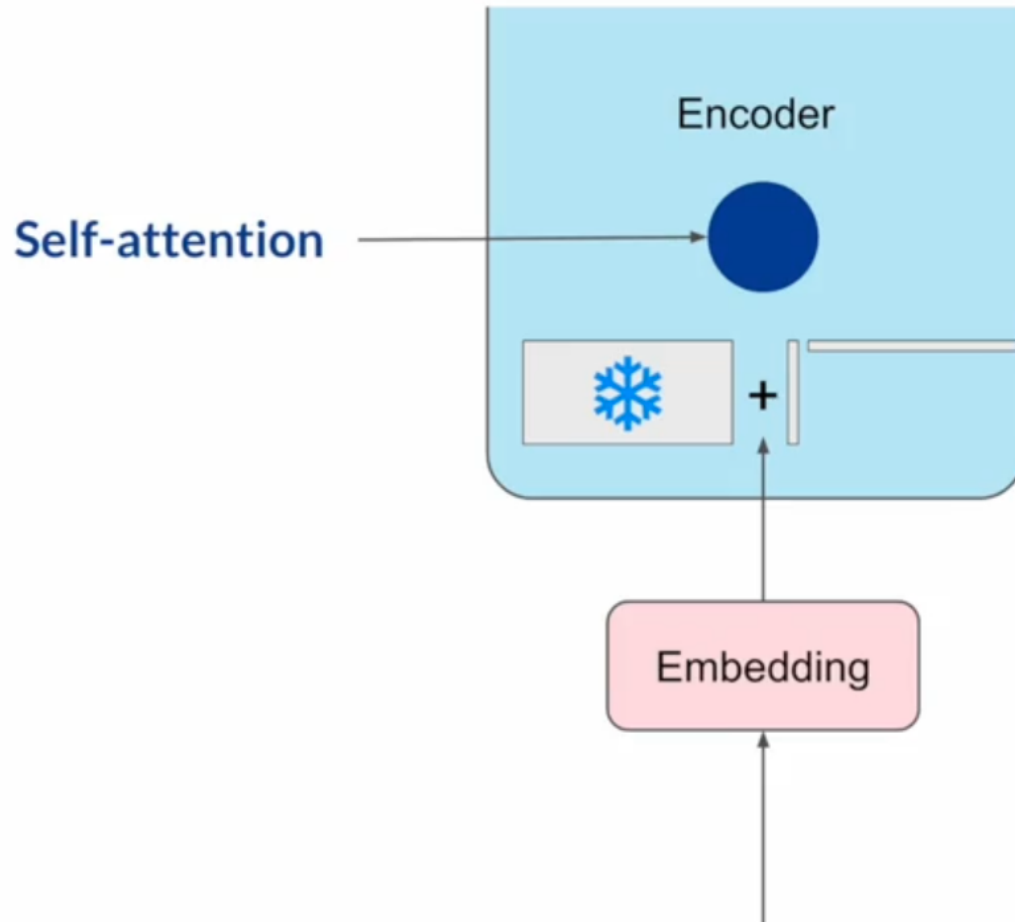
LoRA: Low Rank Adaption of LLMs



1. Freeze most of the original LLM weights.
2. Inject 2 **rank decomposition matrices**



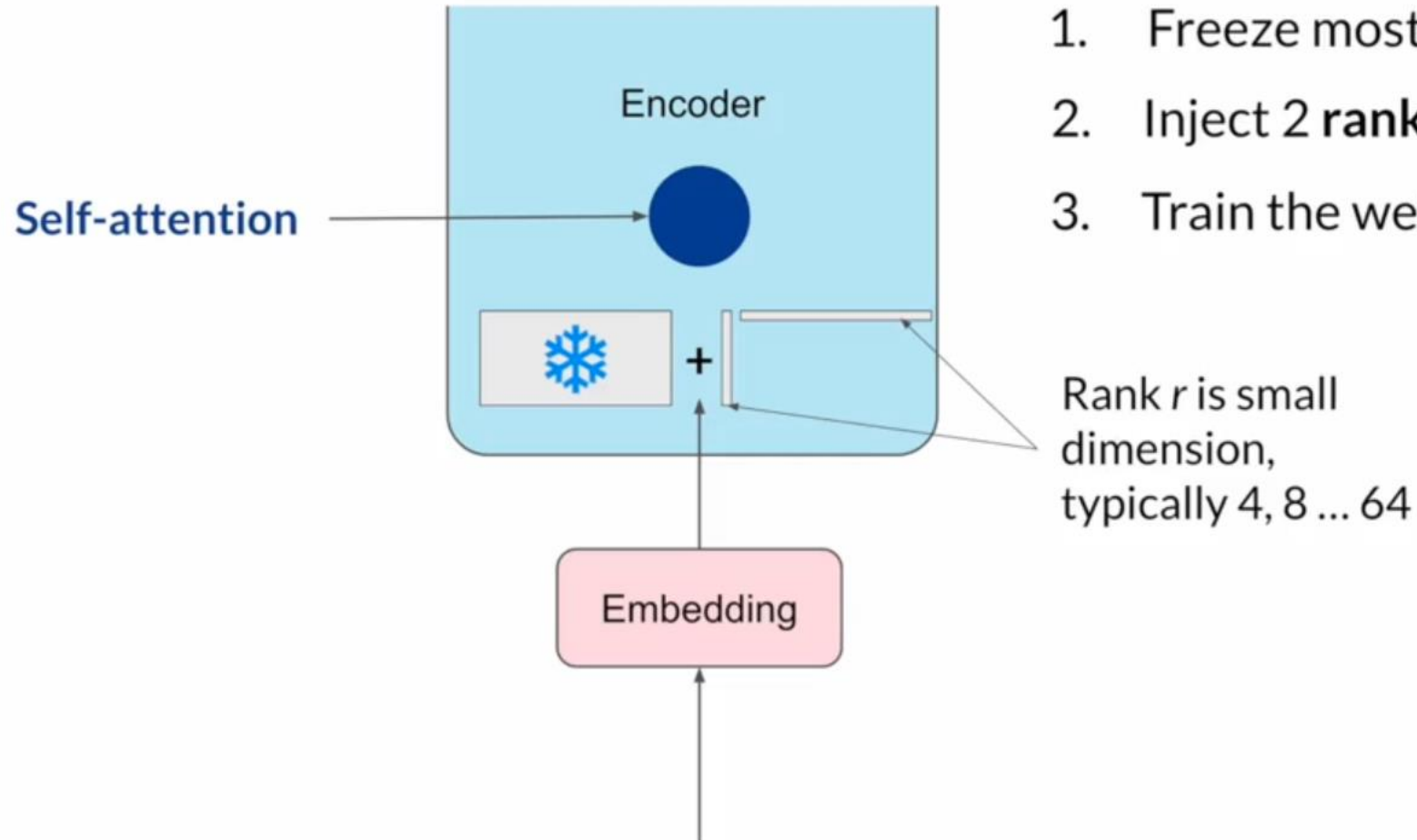
LoRA: Low Rank Adaption of LLMs



1. Freeze most of the original LLM weights.
2. Inject 2 **rank decomposition matrices**
3. Train the weights of the smaller matrices



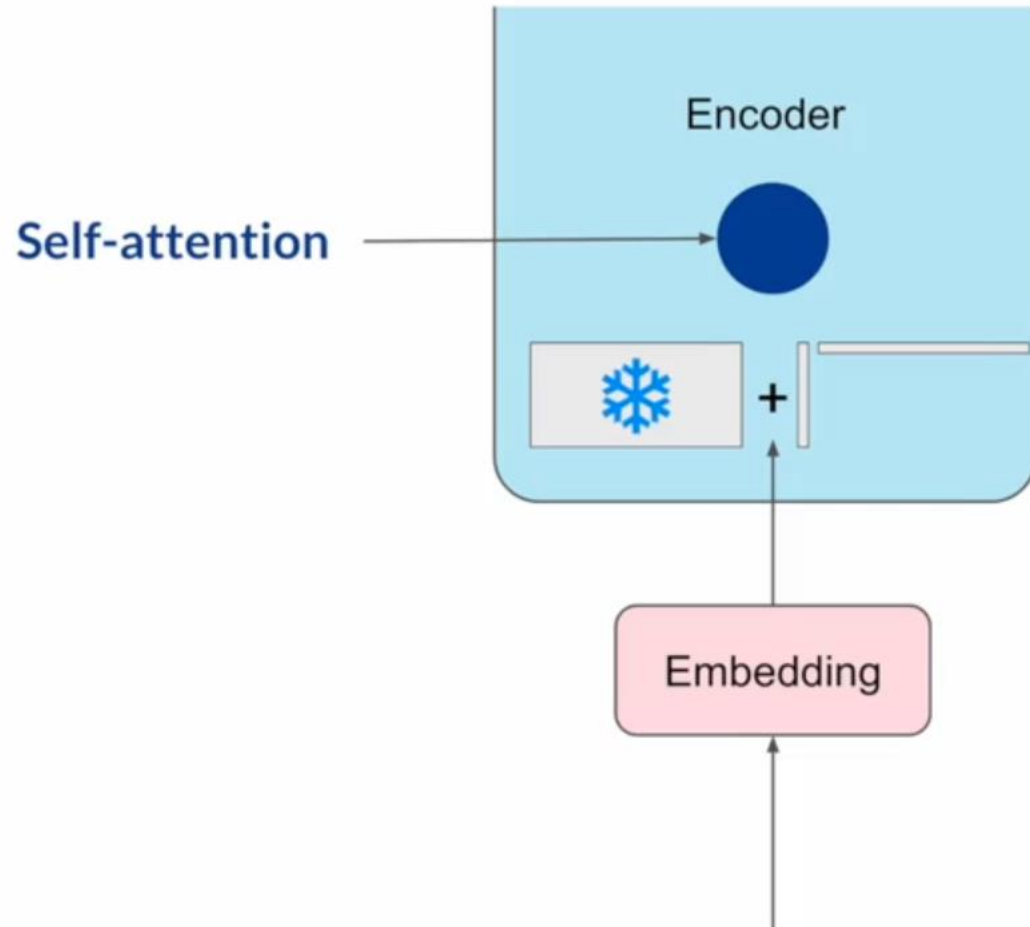
LoRA: Low Rank Adaption of LLMs



1. Freeze most of the original LLM weights.
2. Inject 2 **rank decomposition matrices**
3. Train the weights of the smaller matrices



LoRA: Low Rank Adaption of LLMs



1. Freeze most of the original LLM weights.
2. Inject 2 **rank decomposition matrices**
3. Train the weights of the smaller matrices

Steps to update model for inference

1. Matrix multiply the low rank matrices

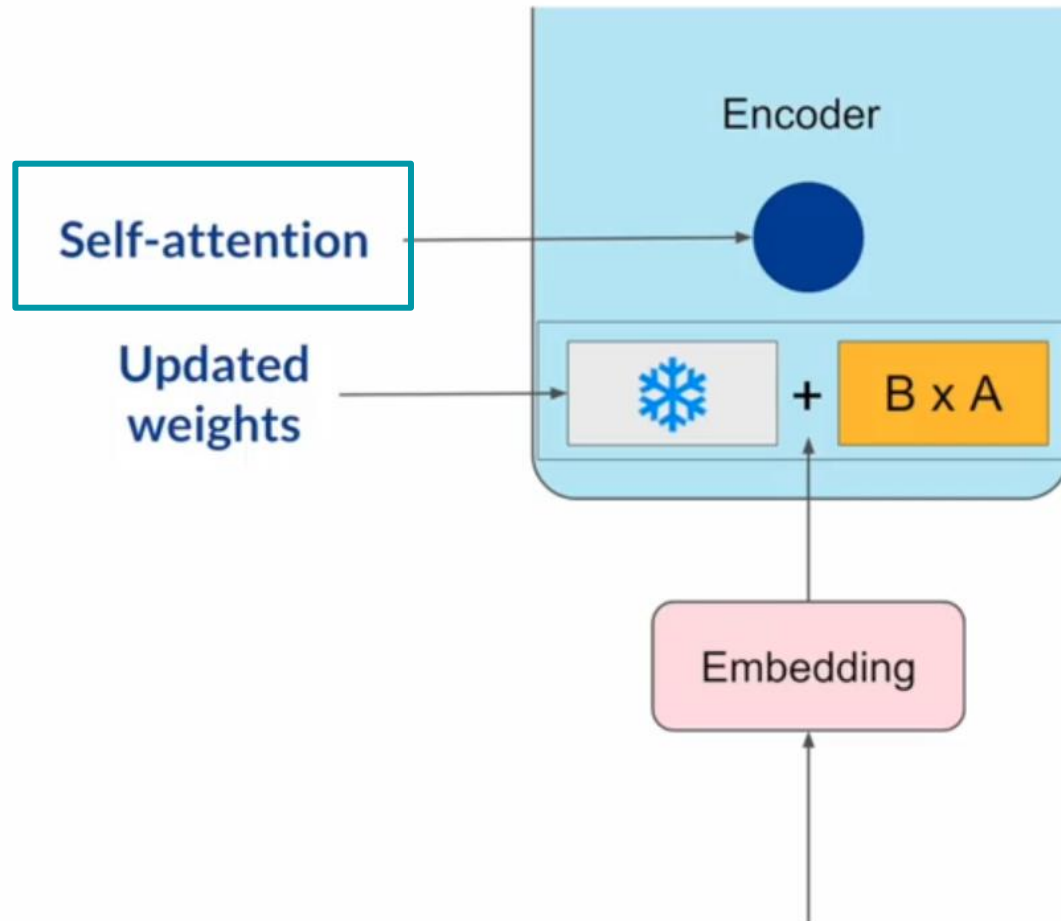
$$\begin{array}{|c|} \hline B \\ \hline \end{array} * \begin{array}{|c|} \hline A \\ \hline \end{array} = \begin{array}{|c|} \hline B \times A \\ \hline \end{array}$$

2. Add to original weights

$$\begin{array}{|c|} \hline \text{Snowflake} \\ \hline \end{array} + \begin{array}{|c|} \hline B \times A \\ \hline \end{array}$$



LoRA: Low Rank Adaption of LLMs



1. Freeze most of the original LLM weights.
2. Inject 2 **rank decomposition matrices**
3. Train the weights of the smaller matrices

Steps to update model for inference:

1. Matrix multiply the low rank matrices

$$\begin{array}{|c|} \hline B \\ \hline \end{array} * \begin{array}{|c|} \hline A \\ \hline \end{array} = \begin{array}{|c|} \hline B \times A \\ \hline \end{array}$$

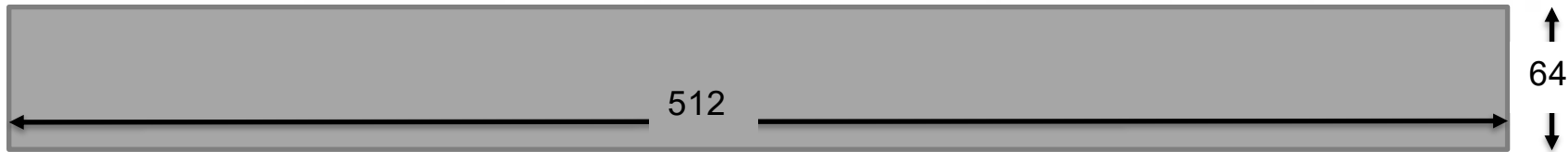
2. Add to original weights

$$\begin{array}{|c|} \hline \text{Snowflake} \\ \hline \end{array} + \begin{array}{|c|} \hline B \times A \\ \hline \end{array}$$



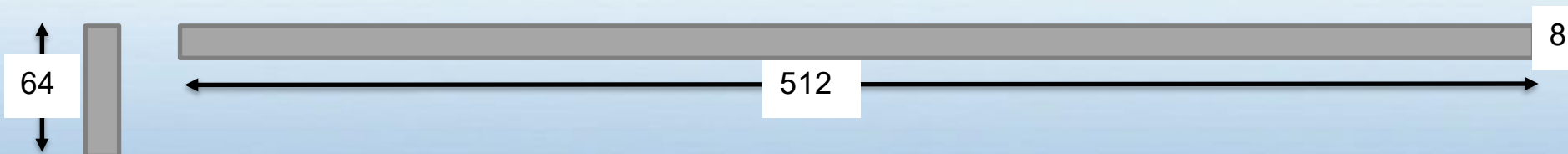
Concrete example using base Transformer as reference

- Use the base Transformer model presented by Vaswani et al. 2017:
 - Transformer weights have dimensions $d \times k = 512 \times 64$
 - So $512 \times 64 = 32,768$ trainable parameters



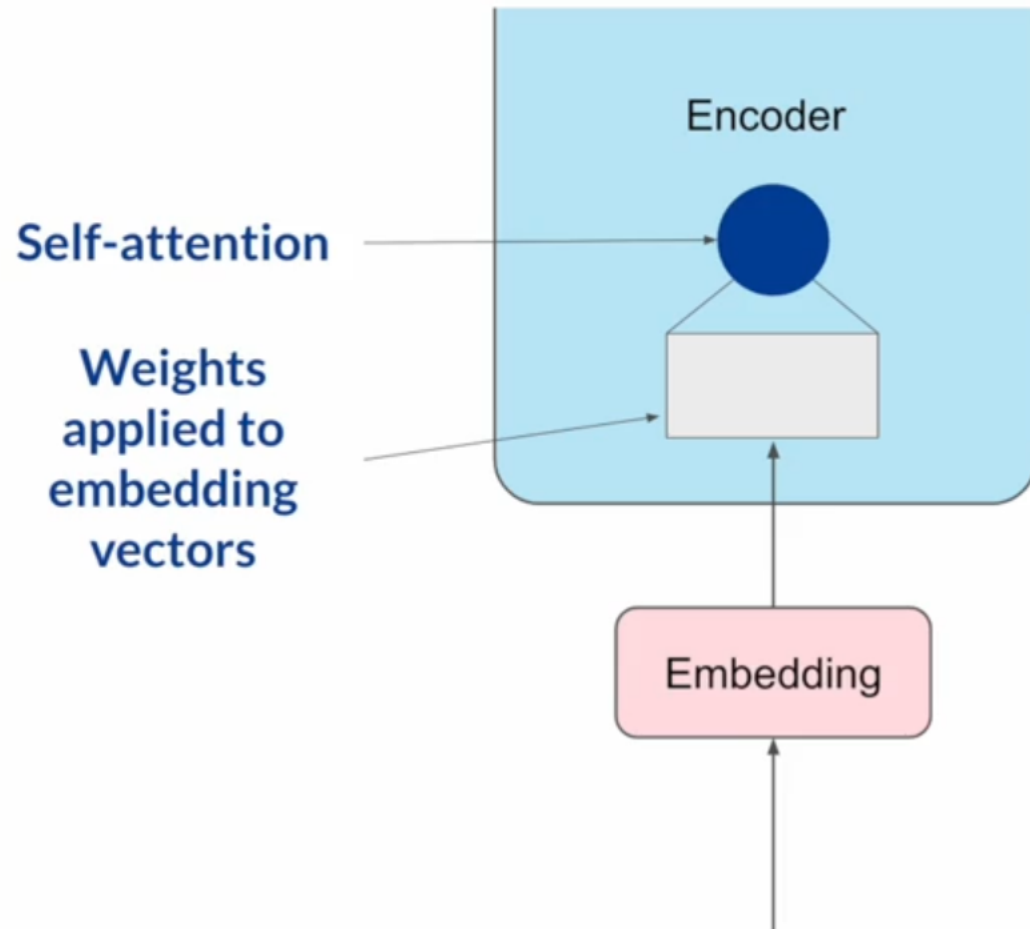
In LoRA with rank $r = 8$:

- A has dimensions $r \times k = 8 \times 64 = 512$ parameters
- B has dimensions $d \times r = 512 \times 8 = 4,096$ parameters
- **86% reduction in parameters to train!**





LoRA: Low Rank Adaption of LLMs



1. Train different rank decomposition matrices for different tasks
2. Update weights before inference

Task A

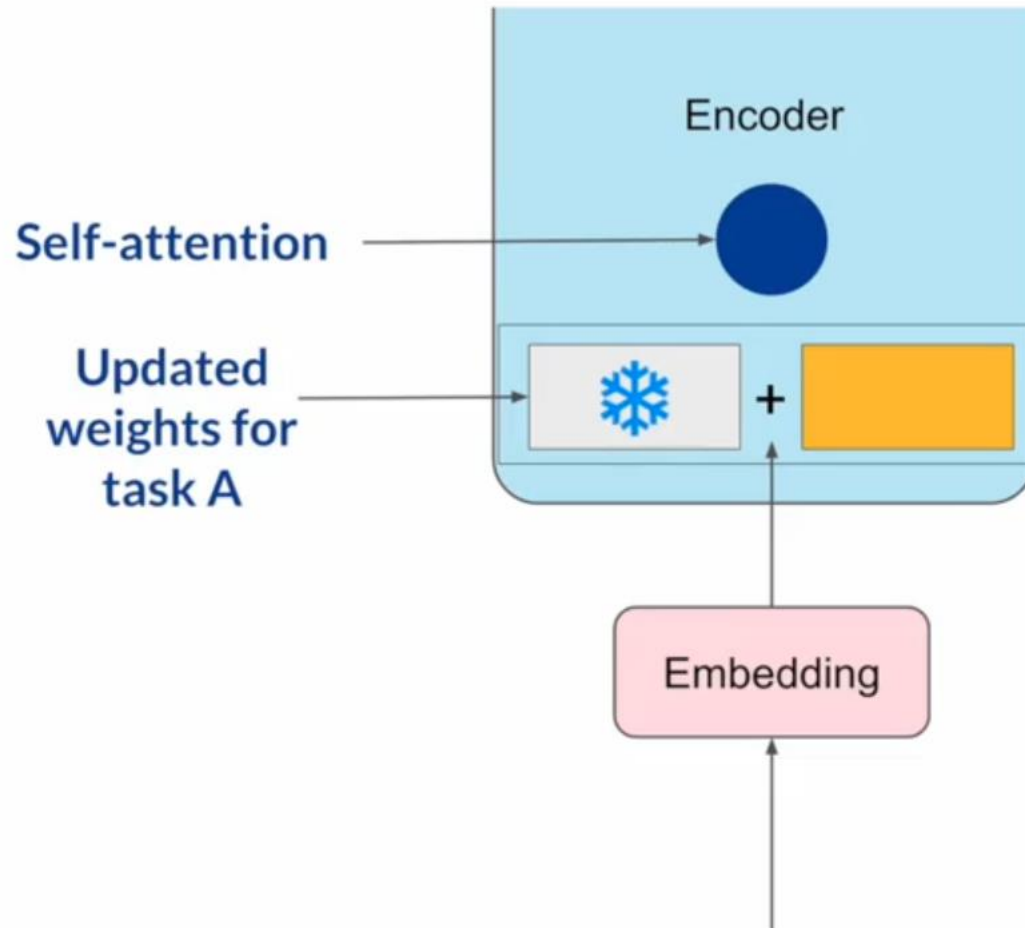
$$\begin{array}{c} \text{orange bar} \\ * \\ \text{orange bar} \end{array} = \text{orange box}$$

Below the equation, a blue box contains the sum of two components:

$$\begin{array}{c} \text{gray box with snowflake} \\ + \\ \text{orange box} \end{array}$$



LoRA: Low Rank Adaption of LLMs



1. Train different rank decomposition matrices for different tasks
2. Update weights before inference

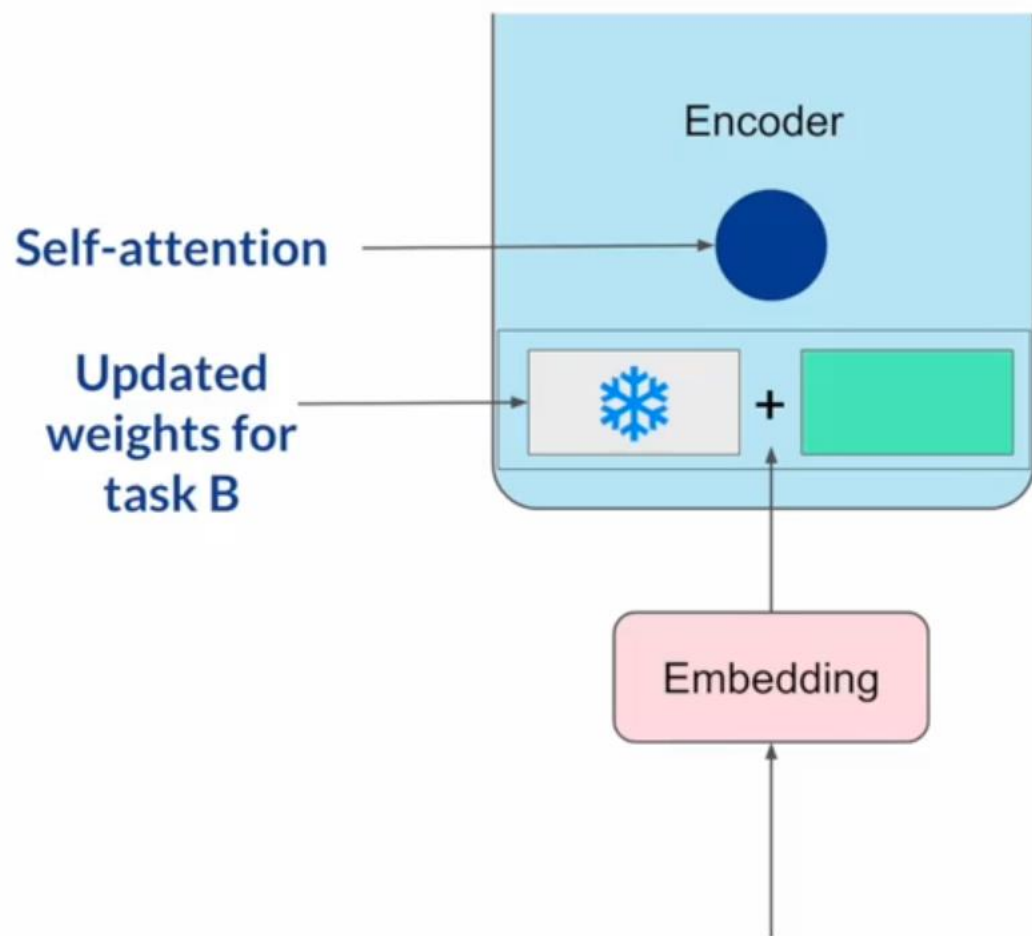
Task A

$$\begin{array}{|c|} \hline \text{orange} \\ \hline \end{array} * \begin{array}{|c|} \hline \text{orange} \\ \hline \end{array} = \begin{array}{|c|} \hline \text{orange} \\ \hline \end{array}$$
$$\begin{array}{|c|} \hline \text{snowflake} \\ \hline \end{array} + \begin{array}{|c|} \hline \text{orange} \\ \hline \end{array}$$

Task B

$$\begin{array}{|c|} \hline \text{green} \\ \hline \end{array} * \begin{array}{|c|} \hline \text{green} \\ \hline \end{array} = \begin{array}{|c|} \hline \text{green} \\ \hline \end{array}$$
$$\begin{array}{|c|} \hline \text{snowflake} \\ \hline \end{array} + \begin{array}{|c|} \hline \text{green} \\ \hline \end{array}$$

LoRA: Low Rank Adaption of LLMs



1. Train different rank decomposition matrices for different tasks
2. Update weights before inference

Task A

$$\begin{array}{c} \text{orange bar} \\ | \end{array} * \begin{array}{c} \text{orange bar} \\ \text{orange bar} \end{array} = \begin{array}{c} \text{orange bar} \\ \text{orange bar} \end{array}$$

$$\begin{array}{c} \text{orange bar} \\ | \end{array} * \begin{array}{c} \text{orange bar} \\ \text{orange bar} \end{array} = \begin{array}{c} \text{orange bar} \\ \text{orange bar} \end{array} + \begin{array}{c} \text{orange bar} \\ \text{orange bar} \end{array}$$

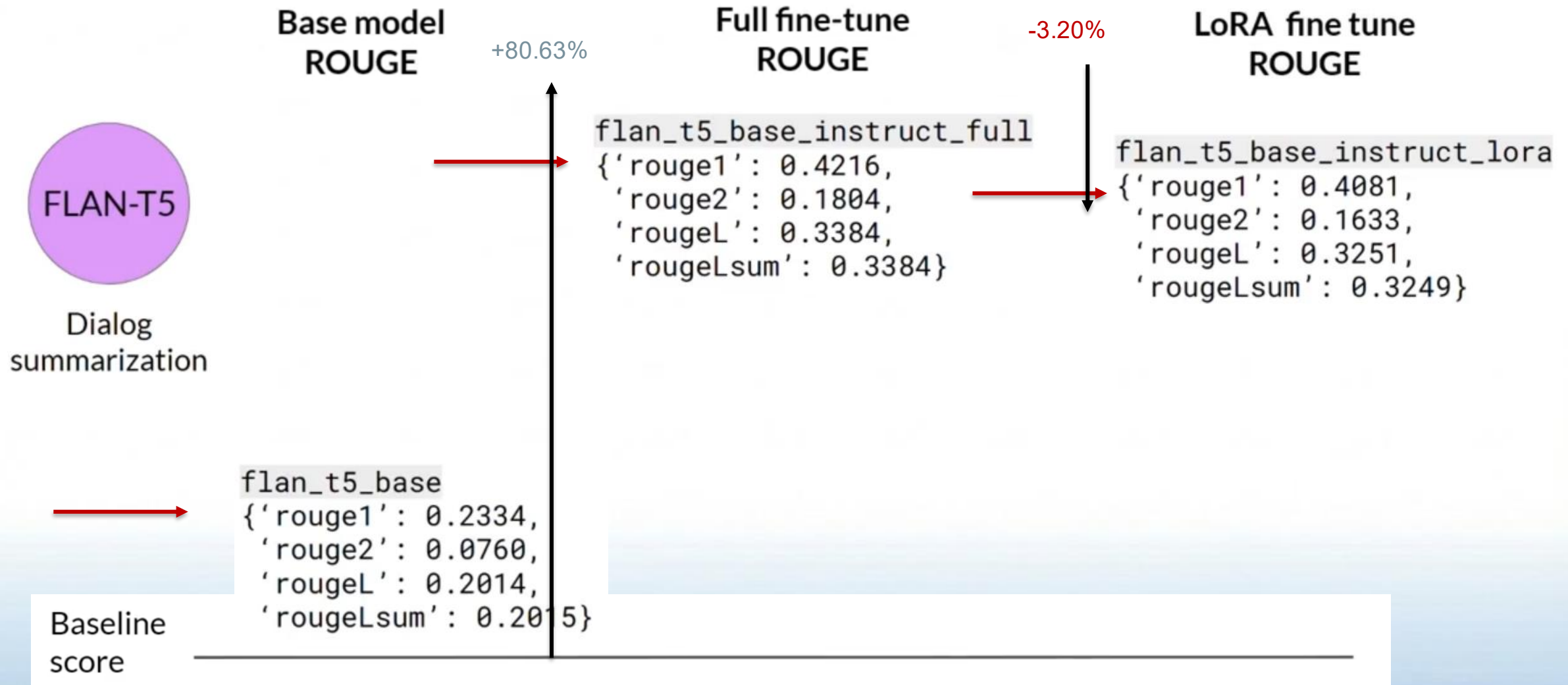
Task B

$$\begin{array}{c} \text{green bar} \\ | \end{array} * \begin{array}{c} \text{green bar} \\ \text{green bar} \end{array} = \begin{array}{c} \text{green bar} \\ \text{green bar} \end{array}$$

$$\begin{array}{c} \text{green bar} \\ | \end{array} * \begin{array}{c} \text{green bar} \\ \text{green bar} \end{array} = \begin{array}{c} \text{green bar} \\ \text{green bar} \end{array} + \begin{array}{c} \text{green bar} \\ \text{green bar} \end{array}$$



Sample ROUGE metrics for full vs. LoRA fine-tuning





Choosing the LoRA rank

Rank r	val_loss	BLEU	NIST	METEOR	ROUGE_L	CIDEr
1	1.23	68.72	8.7215	0.4565	0.7052	2.4329
2	1.21	69.17	8.7413	0.4590	0.7052	2.4639
4	1.18	70.38	8.8439	0.4689	0.7186	2.5349
8	1.17	69.57	8.7457	0.4636	0.7196	2.5196
16	1.16	69.61	8.7483	0.4629	0.7177	2.4985
32	1.16	69.33	8.7736	0.4642	0.7105	2.5255
64	1.16	69.24	8.7174	0.4651	0.7180	2.5070
128	1.16	68.73	8.6718	0.4628	0.7127	2.5030
256	1.16	68.92	8.6982	0.4629	0.7128	2.5012
512	1.16	68.78	8.6857	0.4637	0.7128	2.5025
1024	1.17	69.37	8.7495	0.4659	0.7149	2.5090

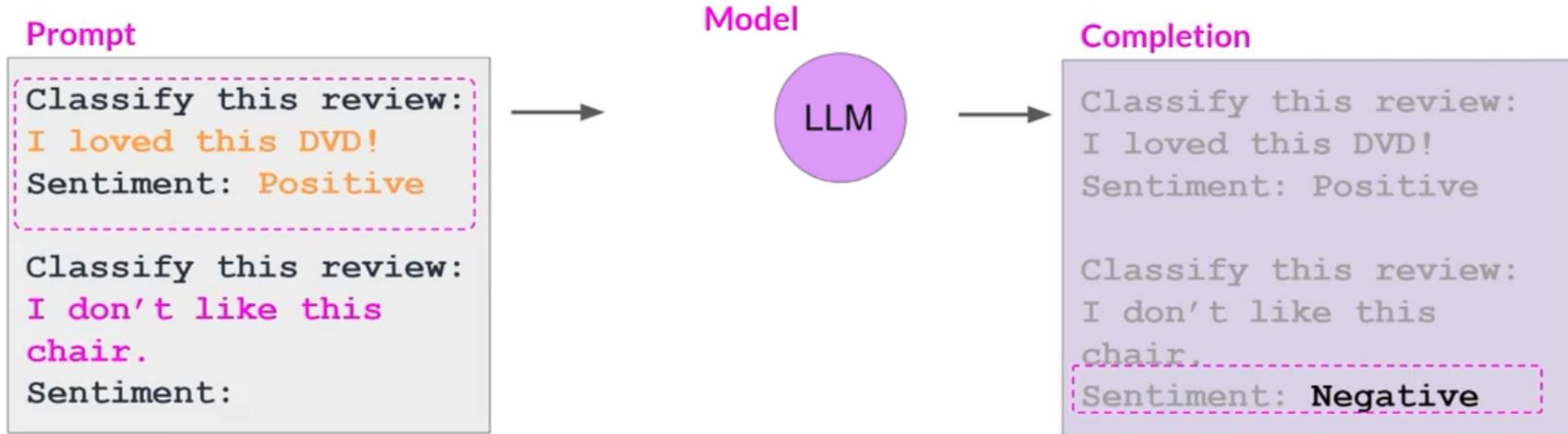
- Effectiveness of higher rank appears to plateau
- Relationship between rank and dataset size needs more empirical data

Prompt tuning with soft prompts





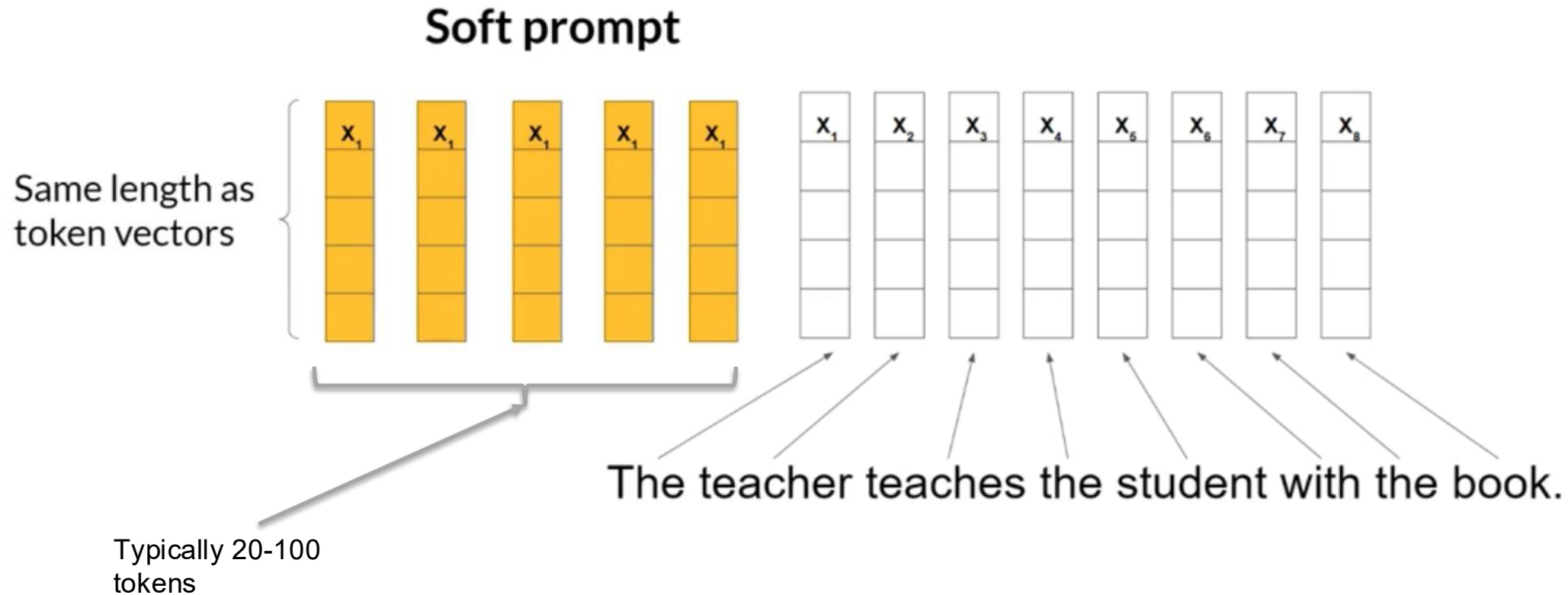
Prompt tuning is **not** prompt engineering!



One-shot or Few-shot Inference

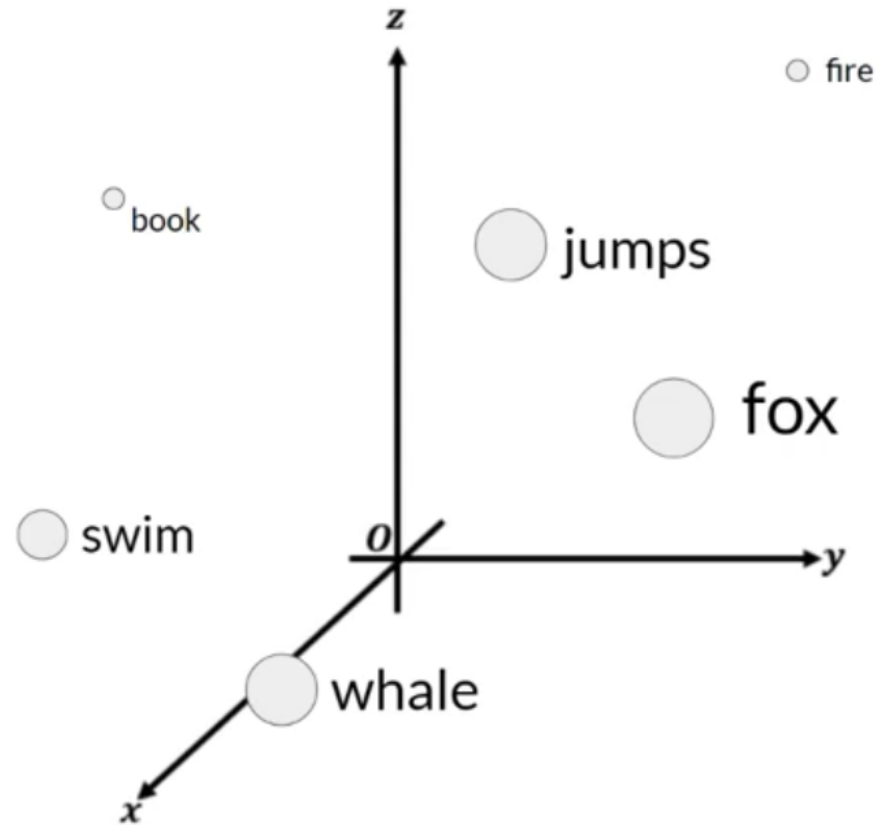


Prompt tuning adds trainable “soft prompt” to inputs





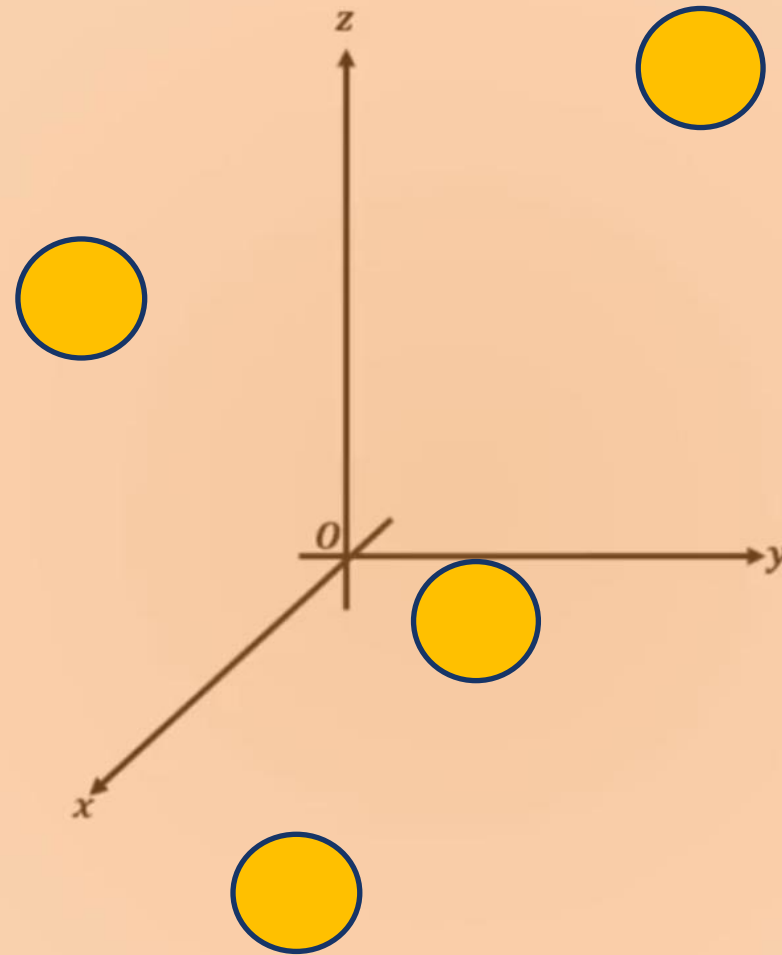
Soft prompts



Embeddings of each token exist at unique point in multi-dimensional space

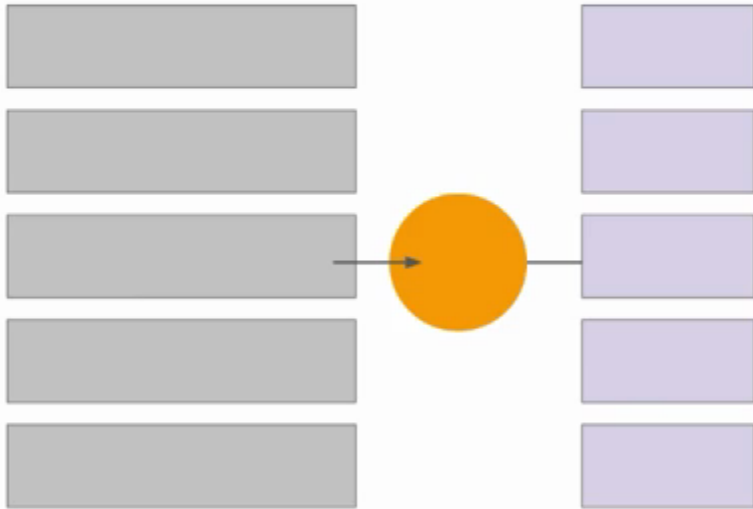


Soft prompts



Full Fine-tuning vs prompt tuning

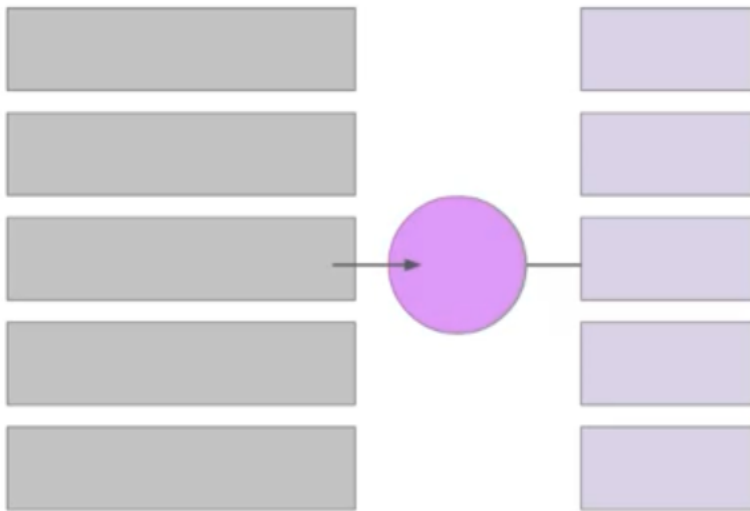
Weights of model updated
during training





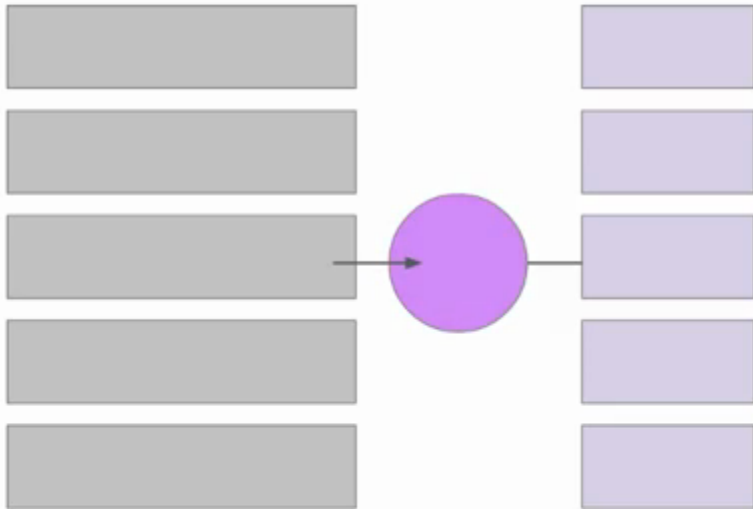
Full Fine-tuning vs prompt tuning

Weights of model updated
during training



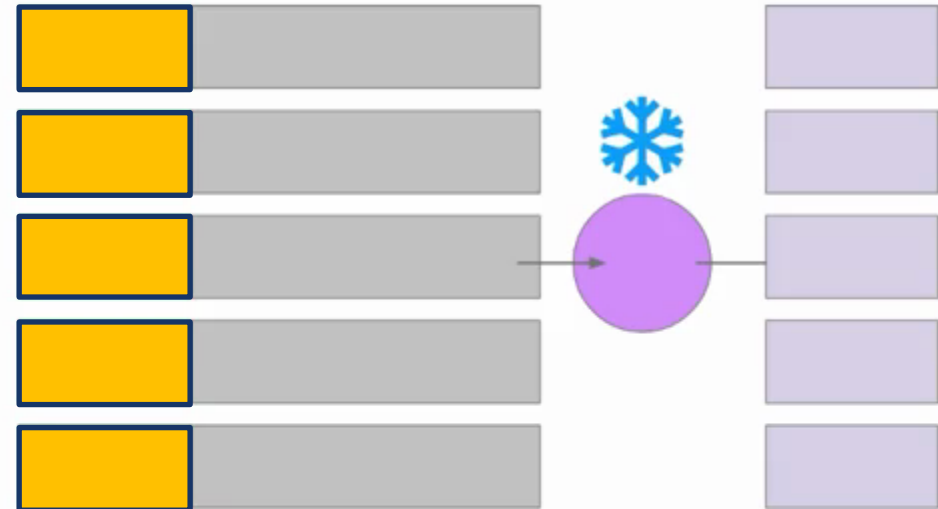
Full Fine-tuning vs prompt tuning

Weights of model updated during training



Millions to Billions of parameter updated

Weights of model frozen and soft prompt trained

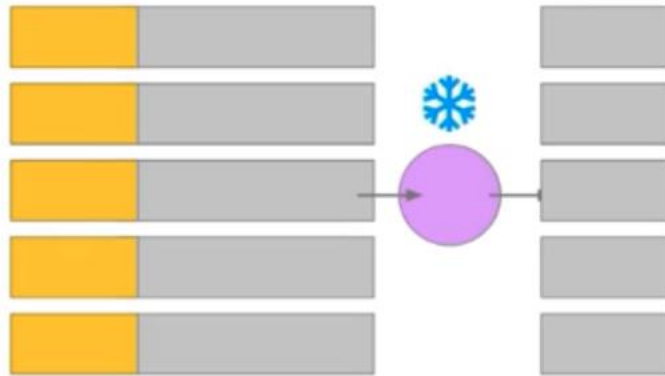


10K – 100K of parameters updated

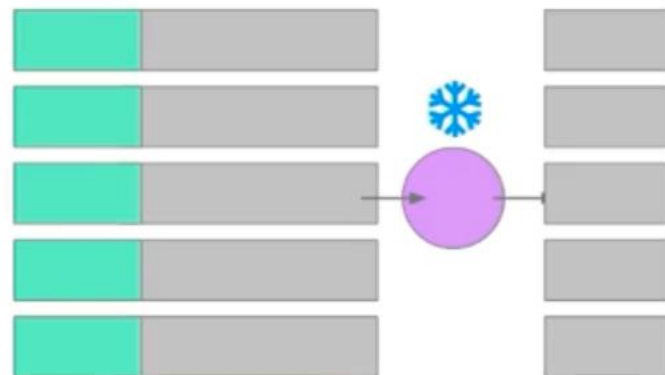


Prompt tuning for multiple tasks

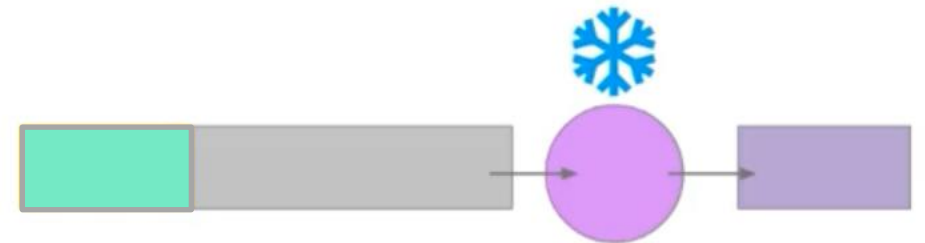
Task A



Task B

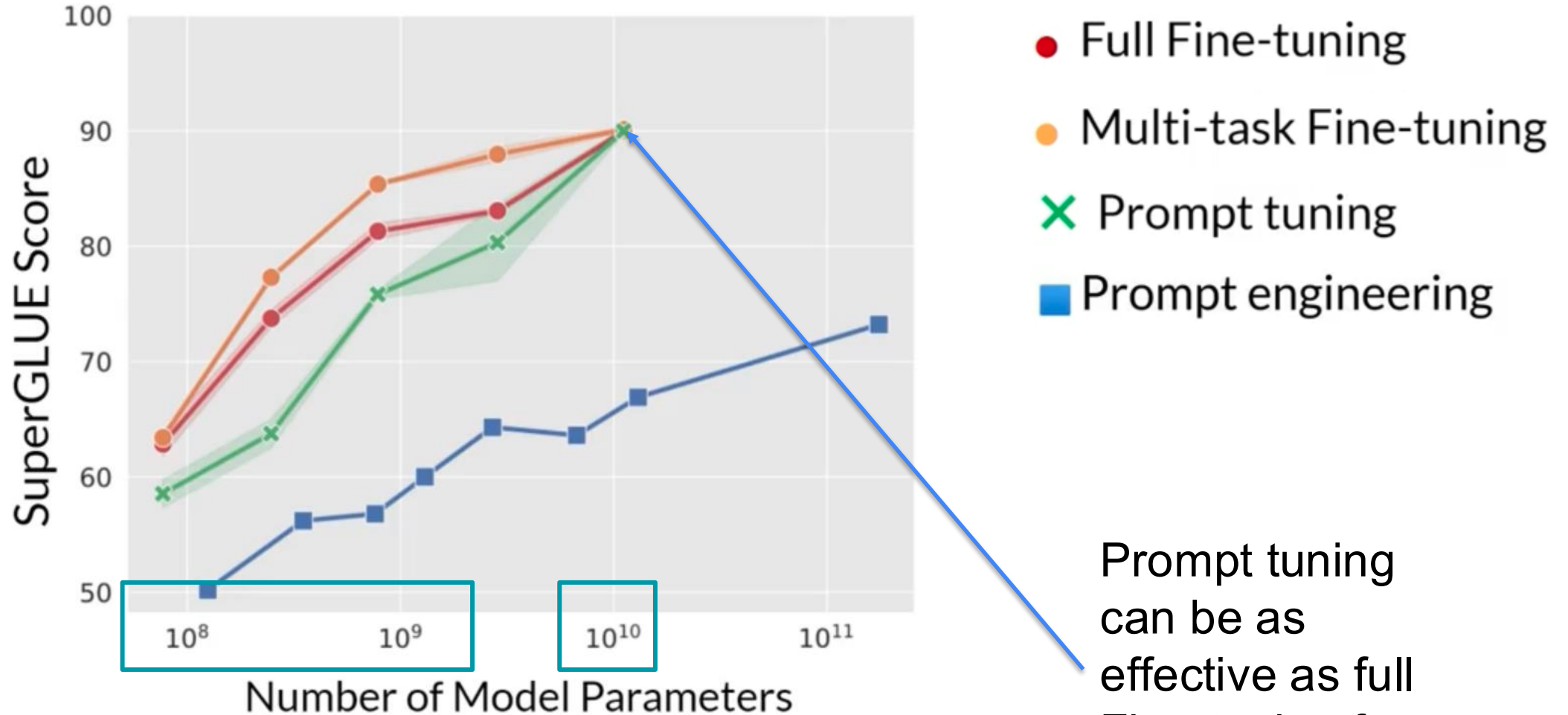


Switch out soft prompt at inference time to change task!





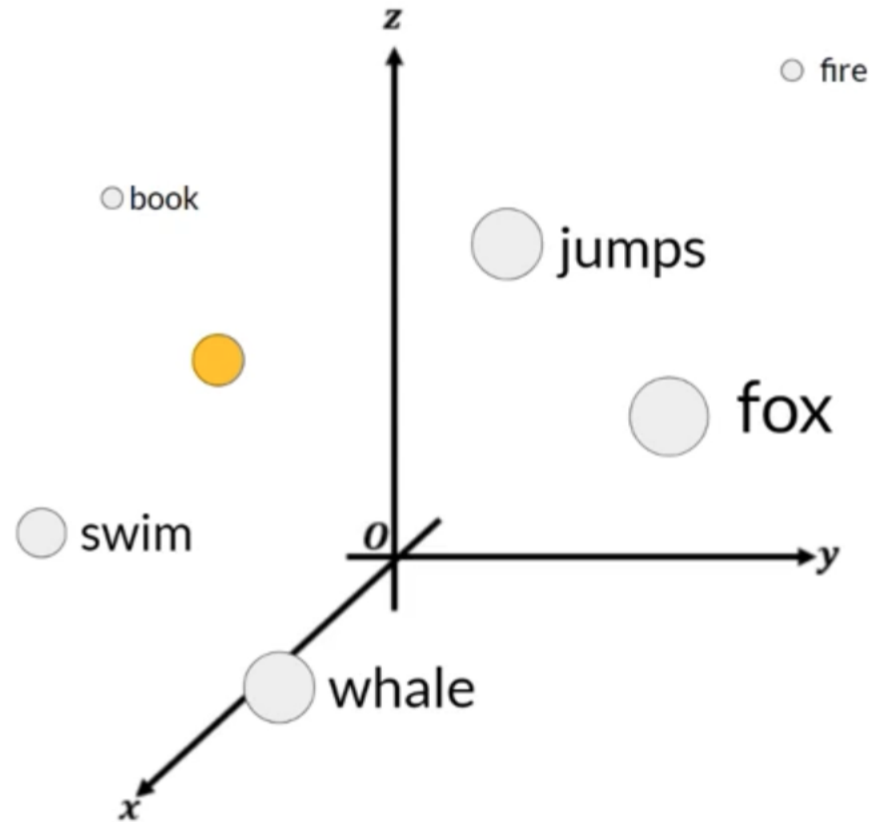
Performance of prompt tuning



Prompt tuning
can be as
effective as full
Fine-tuning for
larger models!



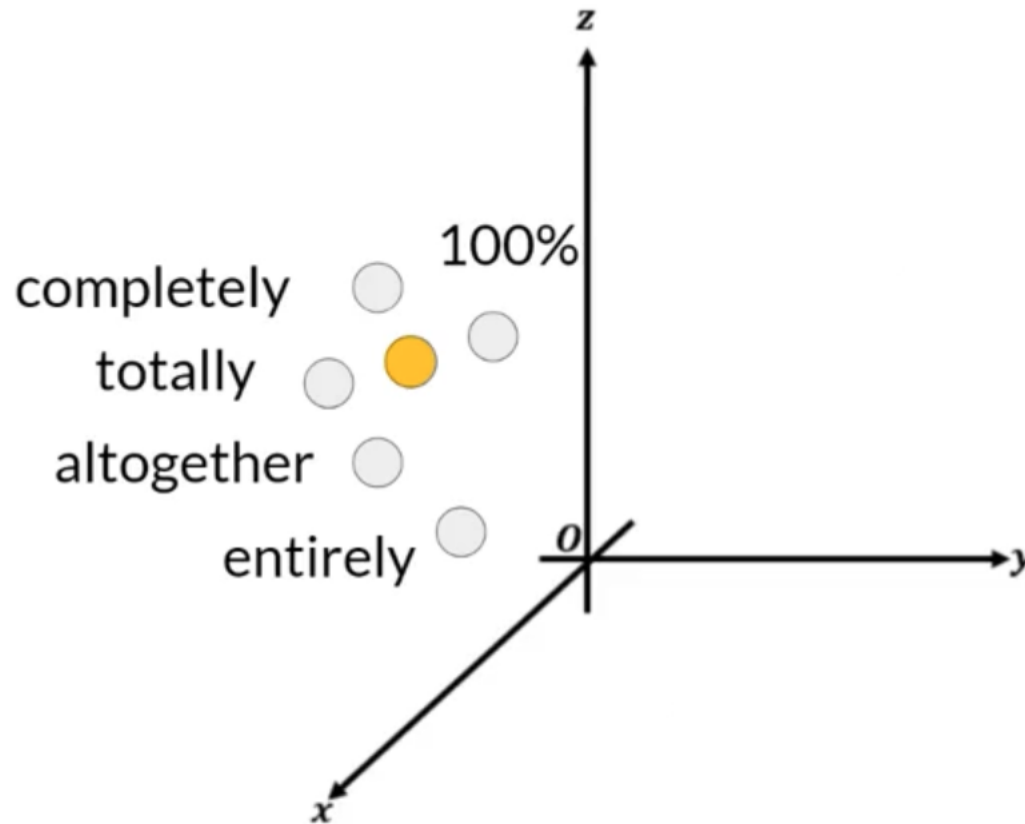
Interpretability of soft prompts



Trained soft-prompt embedding does not correspond to a known token...



Interpretability of soft prompts



...but nearest neighbors
form a semantic group
with similar meanings.

PEFT methods summary

Selective

Select subset of initial LLM parameters to fine-tune

Reparameterization

Reparameterize model weights using a low-rank representation

LoRA

Additive

Add trainable layers or parameters to model

Adapters

Soft Prompts
Prompt Tuning



Key takeaways

- Instruction fine-tuning
 - Prompt templates and data sets for FLAN-T5
- PEFT
 - LoRA
 - Prompt Tuning
- Quantization + LoRA = QLoRA

